

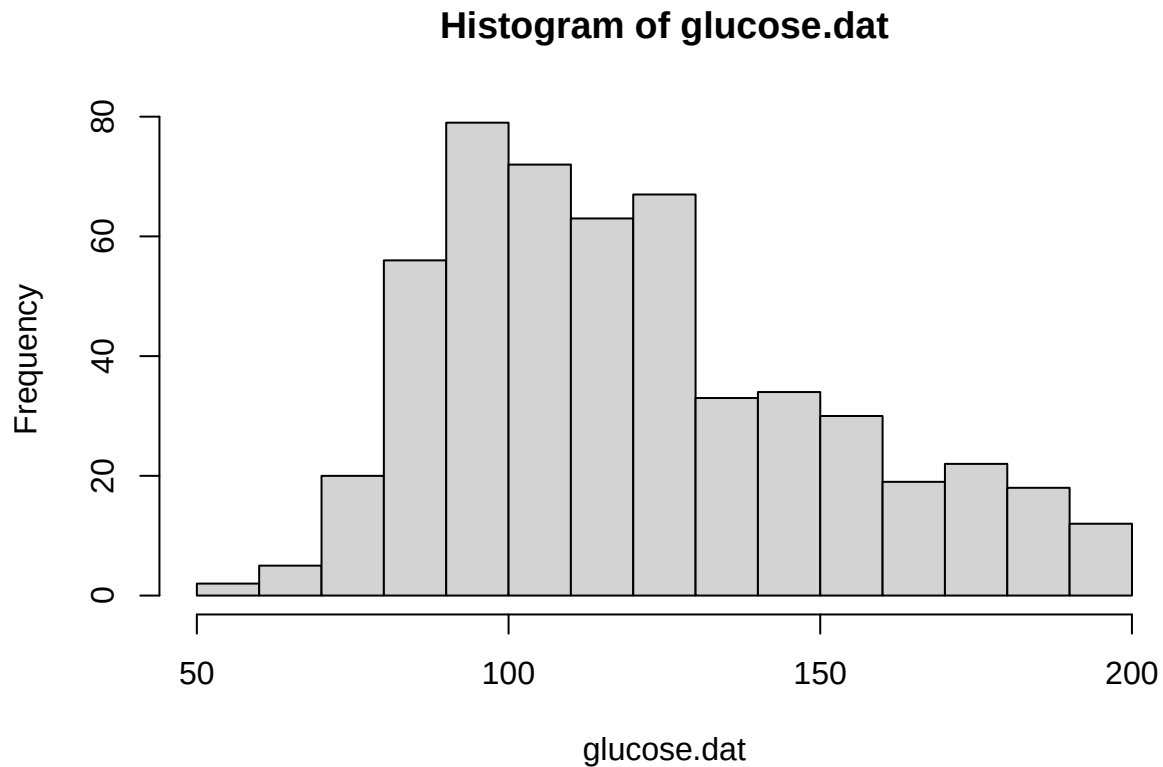
578 HW 5

Nathaniel Pulsone

2025-11-14

Question 1 (a):

```
glucose.dat = read.table("glucose.dat.txt")[,1]
hist(glucose.dat)
```



The empirical distribution deviates from a normal distribution in a number of ways. Primarily, there is significant asymmetry in the distribution due to a rightward skew. In addition, it appears that there could be more than one mode of the distribution (one around 100, possibly another around 150, and possibly an additional one around 175). Finally, although it is difficult to tell based on appearance, the kurtosis of the empirical distribution may differ from that of a normal distribution (the weight of the tails differs).

Question 1 (b): (see pen and paper work at the end)

Question 1 (c): below is code which implements the Gibbs sampler, note that the sampling for part (d) is incorporated into the algorithm

```

# initialize priors
a = 1
b = 1
mu_0 = 120
tao2_0 = 200
sig2_0 = 1000
nu_0 = 10
k_0 = 10

# initialize posteriors for updating

mu_1s = mu_0
mu_2s = mu_0
tao2_1s = tao2_0
tao2_2s = tao2_0
sig2_n1s = sig2_0
sig2_n2s = sig2_0
nu_1s = nu_0
nu_2s = nu_0
k_1s = k_0
k_2s = k_0

# initialize algorithm parameters and samples
Ytild_s = c()
theta_mins = c()
theta_maxs = c()

# initialize sample data metrics
Y = glucose.dat
n = length(Y)

# Gibbs sampler
iters = 10000
for(i in seq(1,iters)){
  p = rbeta(1, a, b) # sample p
  X = rbinom(n, 1, p) # sample x

  # here, 0s are assigned to group 1, and 1s assigned to group 2

  # update parameters related to p, x

  n_1s = sum(1 - X)
  n_2s = n - n_1s

  a = a + n_2s
  b = b + n_1s

  Y1s = Y[which(X == 0)] # observations assigned to group 1
  Y2s = Y[which(X == 1)] # to group 2

  Ybar_1s = mean(Y1s)
  Ybar_2s = mean(Y2s)

```

```

# update group 1 parameters

sig2_n1s = (1 / (nu_0 + n_1s)) * (nu_0*sig2_0 + sum((Y1s - Ybar_1s)^2) + k_0*n_1s/(k_0 + n_1s) *(Ybar_1s - Ybar_0)^2)
nu_1s = nu_0 + n_1s

sig2_1s = 1 / (rgamma(1, nu_1s/2, nu_1s/2 * sig2_n1s)) # sample sig2_1

lambda_0 = 1/tao2_0
lambda_n = lambda_0 + n_1s / sig2_1s

mu_1s = (lambda_0/lambda_n)*mu_0 + ((n_1s / sig2_1s)/lambda_n)*Ybar_1s
tao_1s = 1/lambda_n

theta_1s = rnorm(1, mu_1s, sqrt(tao_1s)) # sample theta_1

# update group 2 parameters

sig2_n2s = (1 / (nu_0 + n_2s)) * (nu_0*sig2_0 + sum((Y2s - Ybar_2s)^2) + k_0*n_2s/(k_0 + n_2s) *(Ybar_2s - Ybar_0)^2)
nu_2s = nu_0 + n_2s

sig2_2s = 1 / (rgamma(1, nu_2s/2, nu_2s/2 * sig2_n2s)) # sample sig2_2

lambda_n = lambda_0 + n_2s / sig2_2s

mu_2s = (lambda_0/lambda_n)*mu_0 + ((n_2s / sig2_2s)/lambda_n)*Ybar_2s
tao_2s = 1/lambda_n

theta_2s = rnorm(1, mu_2s, sqrt(tao_2s)) # sample theta_2

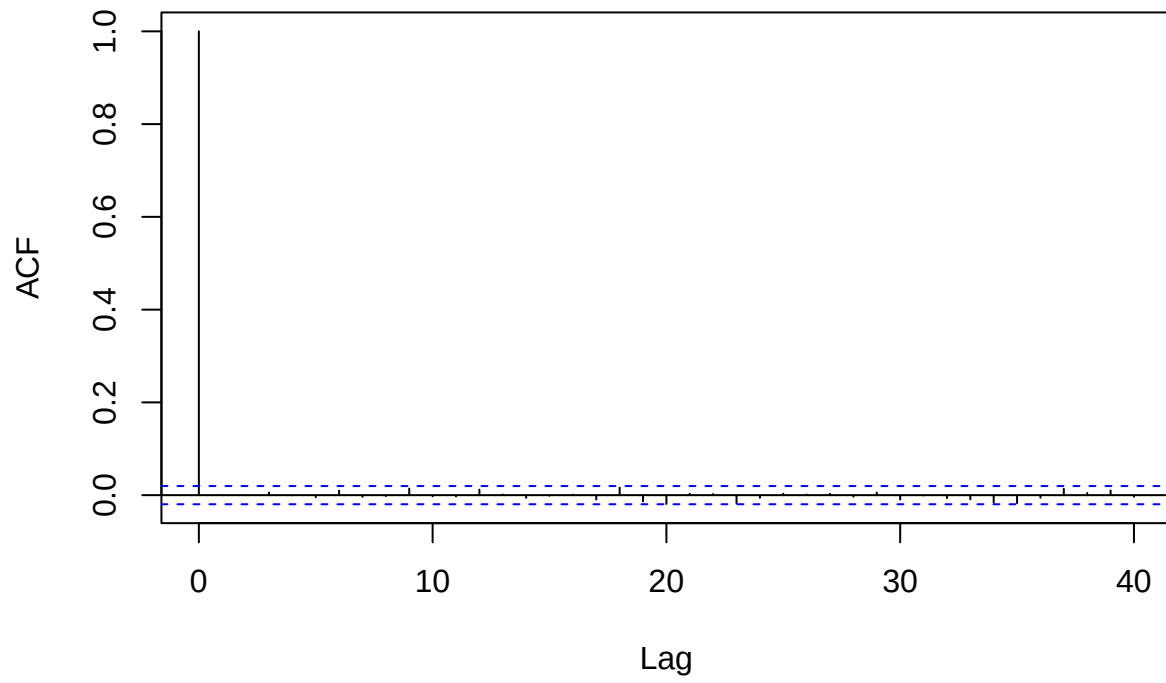
# update theta samples for part (c)
theta_mins = c(theta_mins, min(c(theta_1s, theta_2s)))
theta_maxs = c(theta_maxs, max(c(theta_1s, theta_2s)))

# update Y samples for part (d)
xs = rbinom(1, 1, p)
if(xs == 0){
  Ytild_s = c(Ytild_s, rnorm(1, theta_1s, sqrt(sig2_1s)))
}
if(xs == 1){
  Ytild_s = c(Ytild_s, rnorm(1, theta_2s, sqrt(sig2_2s)))
}
}

acf(theta_maxs)

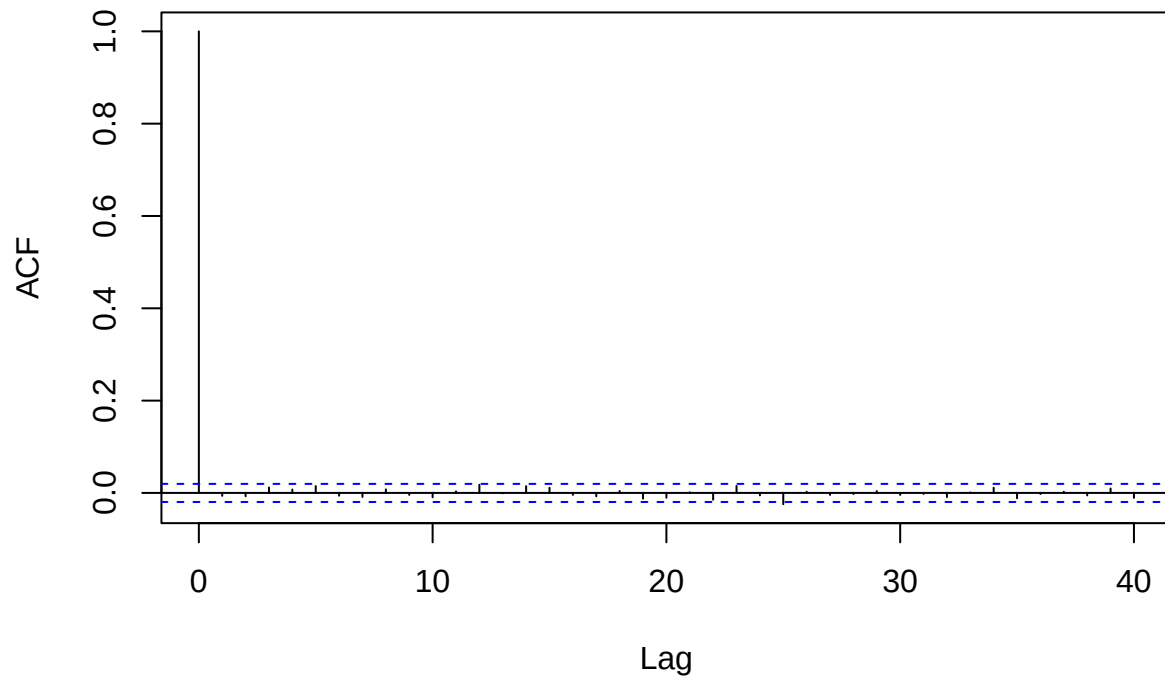
```

Series theta_maxs



```
acf(theta_mins)
```

Series theta_mins



```
effectiveSize(theta_maxs)
```

```
## var1  
## 10000
```

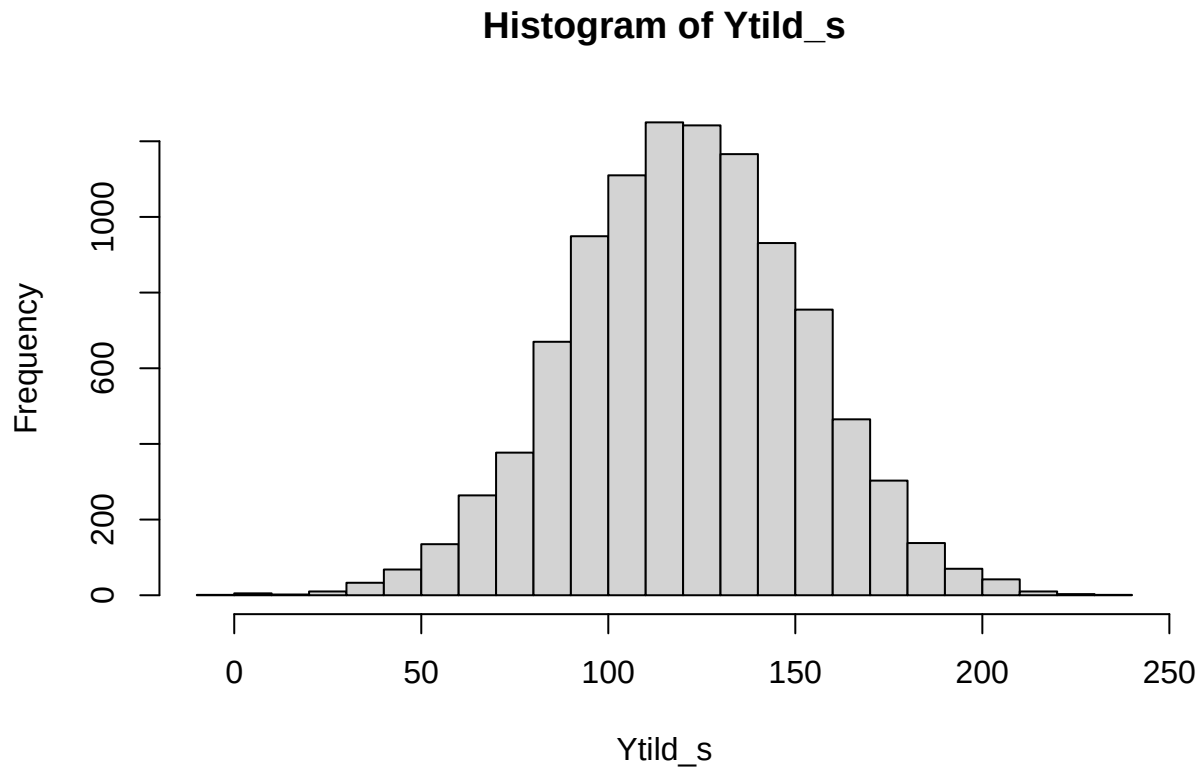
```
effectiveSize(theta_mins)
```

```
## var1  
## 10000
```

Based on the two acf plots, we see that the `theta_mins` series and `theta_maxs` series both have insignificant autocorrelation past lag 0, which seems to imply that the samples from the Gibbs Sampler are close to iid. Accordingly, we see that the effective sample size is computed as essentially equal to the iteration count, 10000.

Question 1 (d): Below is a histogram of the collected samples

```
hist(Ytild_s, breaks = 30)
```



It does not appear as though this mixture model is a completely adequate fit, since the distribution of samples from the Gibbs sampler does not feature the same skew that the empirical distribution has. For the Gibbs Sampler distribution, although there is some rightward skew, we appear to have something close to a Gaussian centered between 100 and 150, while the empirical distribution features more significant rightward skew and possible additional modes at 150 and 175. Since the empirical distribution is made up of only about 500 sample however, it is hard to be certain what the exact shape of the distribution is and the model could very well be adequate.

Question 2 (a):

```
bb_sim <- stan(
  file = "578 HW 5 Q2.stan",
  data = list(alpha = 2, beta = 2, Y = 9, n = 10),
  chains = 4, iter = 2000, warmup = 1000, thin = 1)
```

```
## Warning in readLines(file, warn = TRUE): incomplete final line found on
```

```
## 'C:\Users\njp09\Documents\Notes\6 - Fall 2025\Bayesian Stats\578 HW 5 Q2.stan'
```

```
##
```

```
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 1).
```

```
## Chain 1:
```

```
## Chain 1: Gradient evaluation took 1.5e-05 seconds
```

```
## Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 0.15 seconds.
```

```
## Chain 1: Adjust your expectations accordingly!
```

```
## Chain 1:
```

```
## Chain 1:
```

```

## Chain 1: Iteration:    1 / 2000 [ 0%] (Warmup)
## Chain 1: Iteration:   200 / 2000 [ 10%] (Warmup)
## Chain 1: Iteration:   400 / 2000 [ 20%] (Warmup)
## Chain 1: Iteration:   600 / 2000 [ 30%] (Warmup)
## Chain 1: Iteration:   800 / 2000 [ 40%] (Warmup)
## Chain 1: Iteration:  1000 / 2000 [ 50%] (Warmup)
## Chain 1: Iteration: 1001 / 2000 [ 50%] (Sampling)
## Chain 1: Iteration:  1200 / 2000 [ 60%] (Sampling)
## Chain 1: Iteration:  1400 / 2000 [ 70%] (Sampling)
## Chain 1: Iteration:  1600 / 2000 [ 80%] (Sampling)
## Chain 1: Iteration:  1800 / 2000 [ 90%] (Sampling)
## Chain 1: Iteration:  2000 / 2000 [100%] (Sampling)
## Chain 1:
## Chain 1: Elapsed Time: 0.022 seconds (Warm-up)
## Chain 1:                0.024 seconds (Sampling)
## Chain 1:                0.046 seconds (Total)
## Chain 1:
##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 2).
## Chain 2:
## Chain 2: Gradient evaluation took 5e-06 seconds
## Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 0.05 seconds.
## Chain 2: Adjust your expectations accordingly!
## Chain 2:
## Chain 2:
## Chain 2: Iteration:    1 / 2000 [ 0%] (Warmup)
## Chain 2: Iteration:   200 / 2000 [ 10%] (Warmup)
## Chain 2: Iteration:   400 / 2000 [ 20%] (Warmup)
## Chain 2: Iteration:   600 / 2000 [ 30%] (Warmup)
## Chain 2: Iteration:   800 / 2000 [ 40%] (Warmup)
## Chain 2: Iteration:  1000 / 2000 [ 50%] (Warmup)
## Chain 2: Iteration: 1001 / 2000 [ 50%] (Sampling)
## Chain 2: Iteration:  1200 / 2000 [ 60%] (Sampling)
## Chain 2: Iteration:  1400 / 2000 [ 70%] (Sampling)
## Chain 2: Iteration:  1600 / 2000 [ 80%] (Sampling)
## Chain 2: Iteration:  1800 / 2000 [ 90%] (Sampling)
## Chain 2: Iteration:  2000 / 2000 [100%] (Sampling)
## Chain 2:
## Chain 2: Elapsed Time: 0.017 seconds (Warm-up)
## Chain 2:                0.019 seconds (Sampling)
## Chain 2:                0.036 seconds (Total)
## Chain 2:
##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 3).
## Chain 3:
## Chain 3: Gradient evaluation took 5e-06 seconds
## Chain 3: 1000 transitions using 10 leapfrog steps per transition would take 0.05 seconds.
## Chain 3: Adjust your expectations accordingly!
## Chain 3:
## Chain 3:
## Chain 3: Iteration:    1 / 2000 [ 0%] (Warmup)
## Chain 3: Iteration:   200 / 2000 [ 10%] (Warmup)
## Chain 3: Iteration:   400 / 2000 [ 20%] (Warmup)
## Chain 3: Iteration:   600 / 2000 [ 30%] (Warmup)

```

```

## Chain 3: Iteration: 800 / 2000 [ 40%] (Warmup)
## Chain 3: Iteration: 1000 / 2000 [ 50%] (Warmup)
## Chain 3: Iteration: 1001 / 2000 [ 50%] (Sampling)
## Chain 3: Iteration: 1200 / 2000 [ 60%] (Sampling)
## Chain 3: Iteration: 1400 / 2000 [ 70%] (Sampling)
## Chain 3: Iteration: 1600 / 2000 [ 80%] (Sampling)
## Chain 3: Iteration: 1800 / 2000 [ 90%] (Sampling)
## Chain 3: Iteration: 2000 / 2000 [100%] (Sampling)
## Chain 3:
## Chain 3: Elapsed Time: 0.017 seconds (Warm-up)
## Chain 3: 0.016 seconds (Sampling)
## Chain 3: 0.033 seconds (Total)
## Chain 3:
##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 4).
## Chain 4:
## Chain 4: Gradient evaluation took 6e-06 seconds
## Chain 4: 1000 transitions using 10 leapfrog steps per transition would take 0.06 seconds.
## Chain 4: Adjust your expectations accordingly!
## Chain 4:
## Chain 4:
## Chain 4: Iteration: 1 / 2000 [ 0%] (Warmup)
## Chain 4: Iteration: 200 / 2000 [ 10%] (Warmup)
## Chain 4: Iteration: 400 / 2000 [ 20%] (Warmup)
## Chain 4: Iteration: 600 / 2000 [ 30%] (Warmup)
## Chain 4: Iteration: 800 / 2000 [ 40%] (Warmup)
## Chain 4: Iteration: 1000 / 2000 [ 50%] (Warmup)
## Chain 4: Iteration: 1001 / 2000 [ 50%] (Sampling)
## Chain 4: Iteration: 1200 / 2000 [ 60%] (Sampling)
## Chain 4: Iteration: 1400 / 2000 [ 70%] (Sampling)
## Chain 4: Iteration: 1600 / 2000 [ 80%] (Sampling)
## Chain 4: Iteration: 1800 / 2000 [ 90%] (Sampling)
## Chain 4: Iteration: 2000 / 2000 [100%] (Sampling)
## Chain 4:
## Chain 4: Elapsed Time: 0.017 seconds (Warm-up)
## Chain 4: 0.015 seconds (Sampling)
## Chain 4: 0.032 seconds (Total)
## Chain 4:

```

```

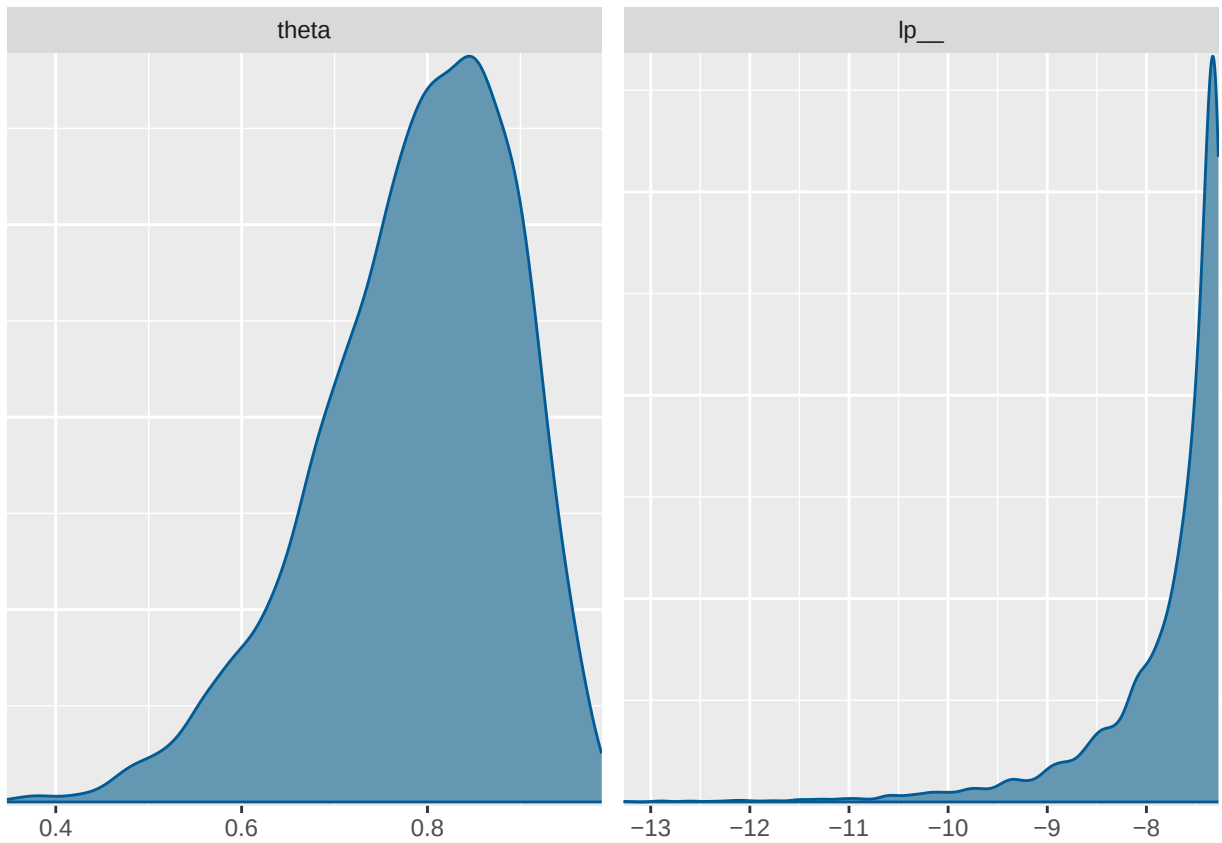
posterior_theta <- as.array(bb_sim, pars = "theta")
dim(posterior_theta)

```

```
## [1] 1000 4 1
```

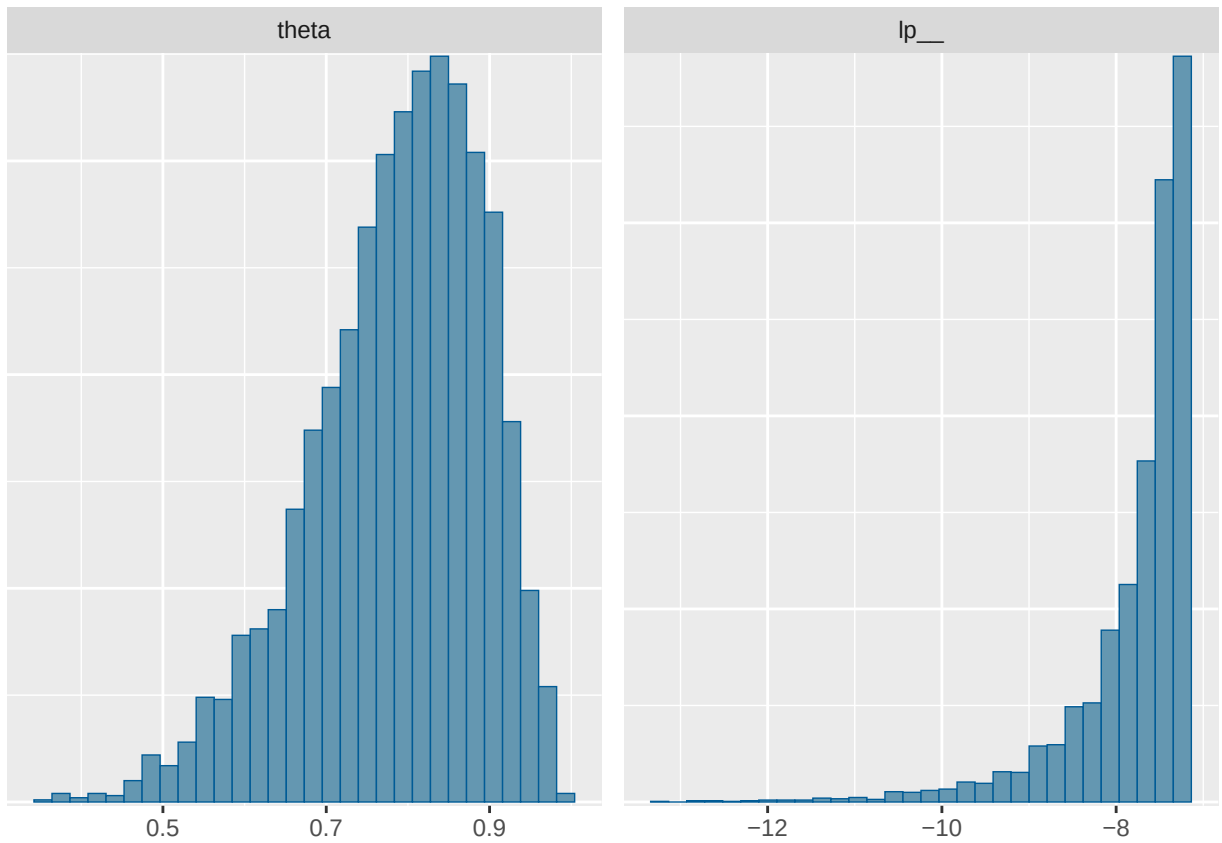
Chains refers to the number of independent Markov chains that are run in parallel for the Bayesian model. Warmup refers to the number of iterations that are counted as the burn in period, meaning the first 1000 iterations are essentially thrown out. Therefore, we have 4 chains, each with a burn-in period of 1000 iterations, and since we have 2000 total iterations for each chain, the total Markov chain sample size will be 4000.

```
mcmc_dens(bb_sim)
```



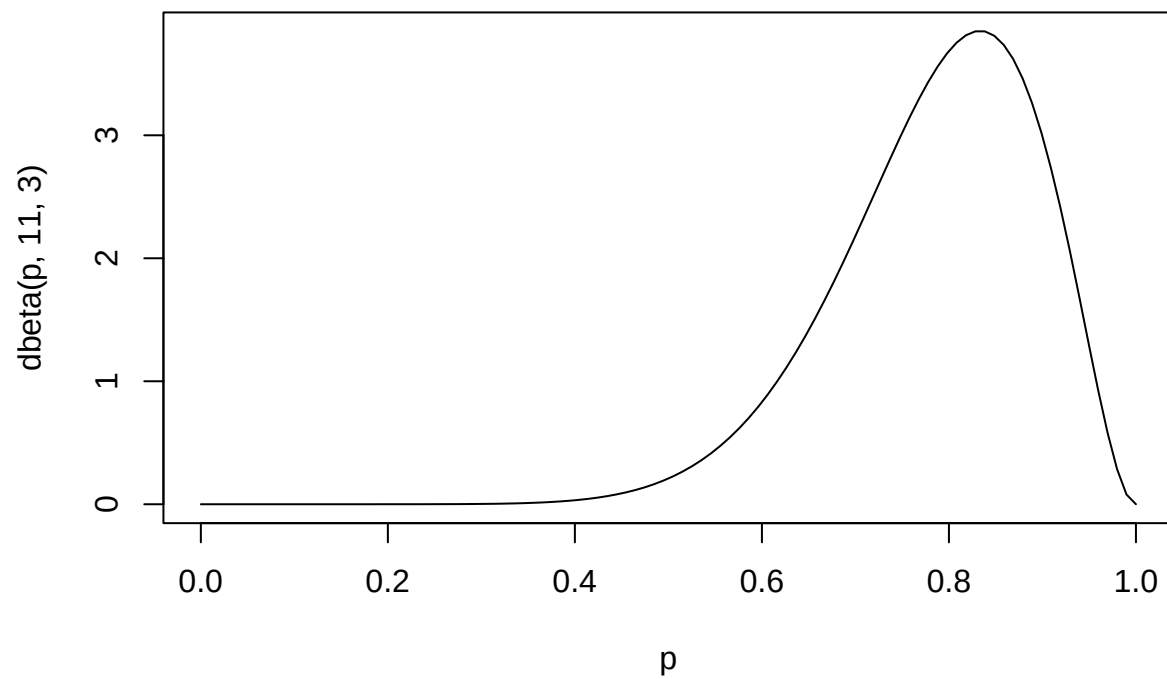
```
mcmc_hist(bb_sim)
```

```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



above is the posterior from the Rstan model, below is the density plot of a $\text{beta}(11, 3)$

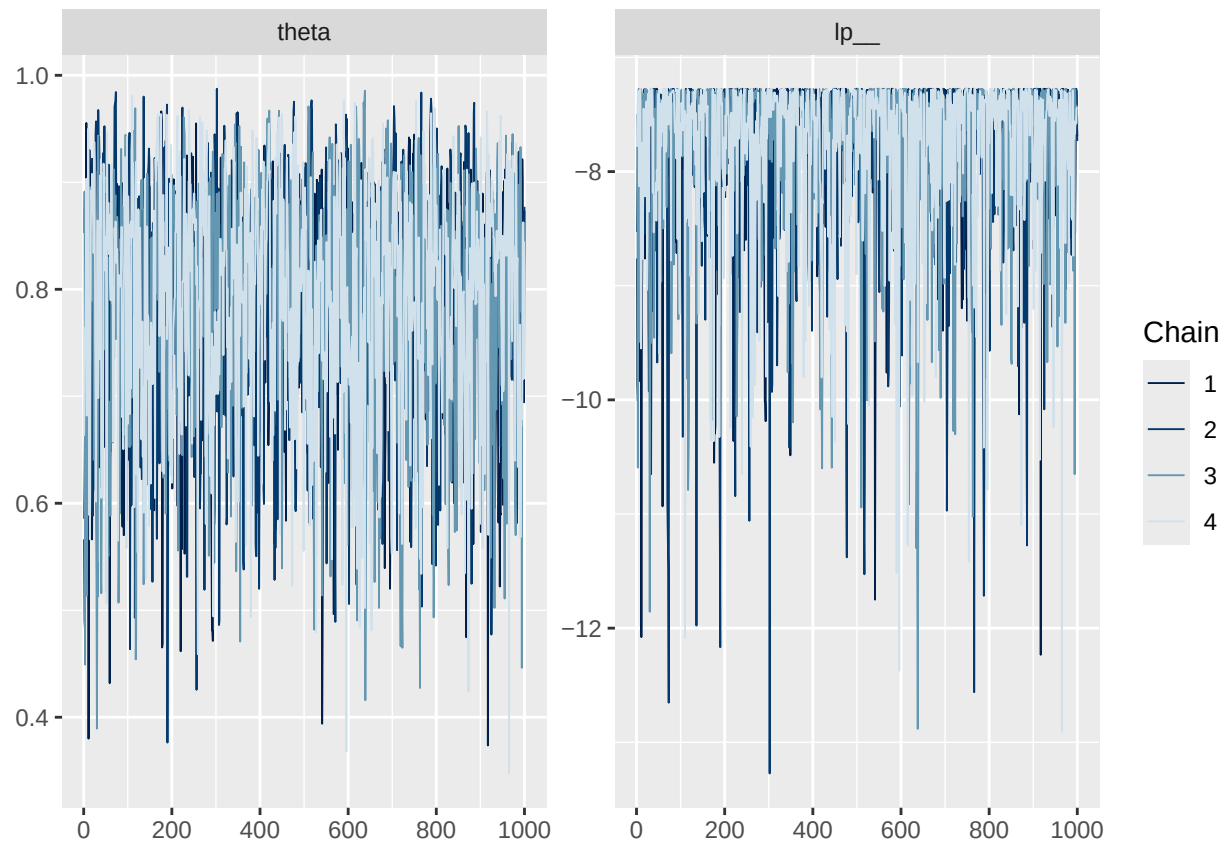
```
p = seq(0, 1, length=100)
plot(p, dbeta(p, 11, 3), type='l')
```



As we can see, the distributions looks nearly the same as expected.

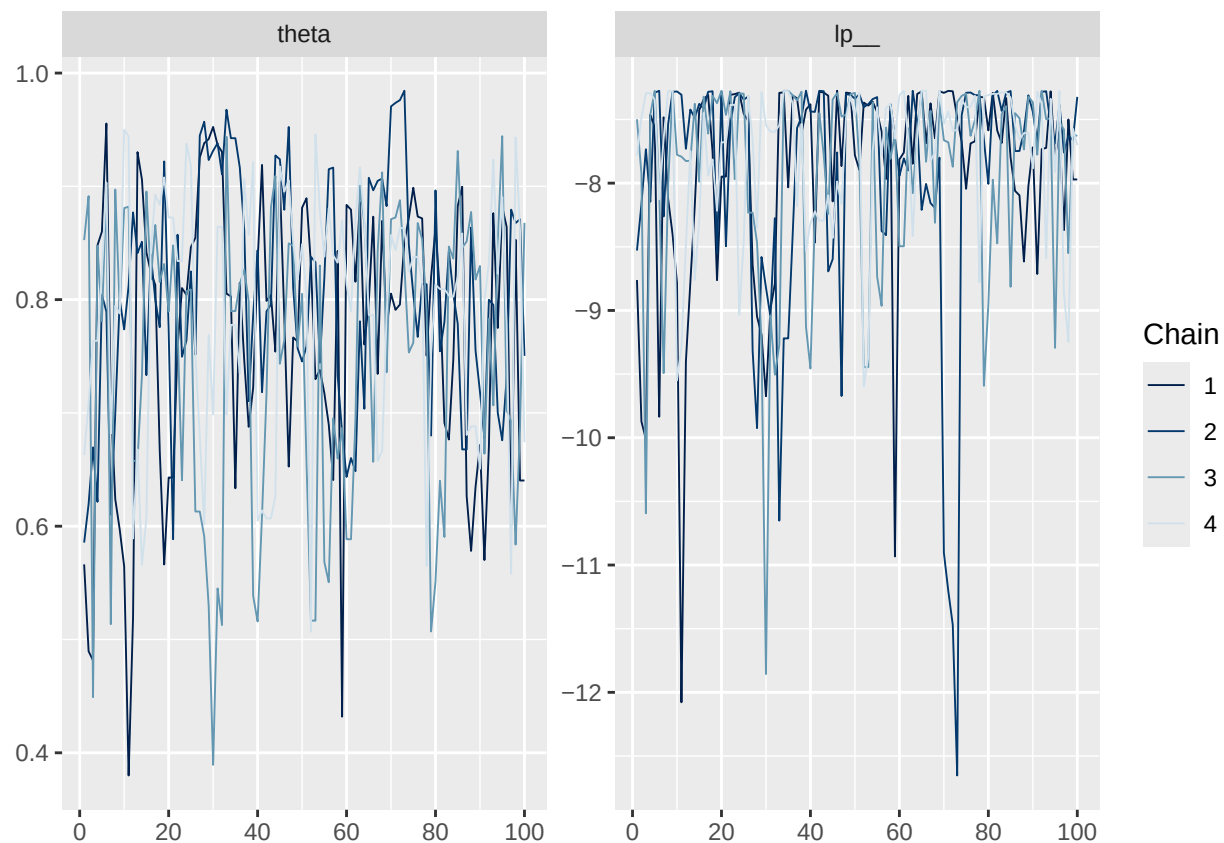
Below is a trace plot for all 4 MCMC chains

```
mcmc_trace(bb_sim)
```



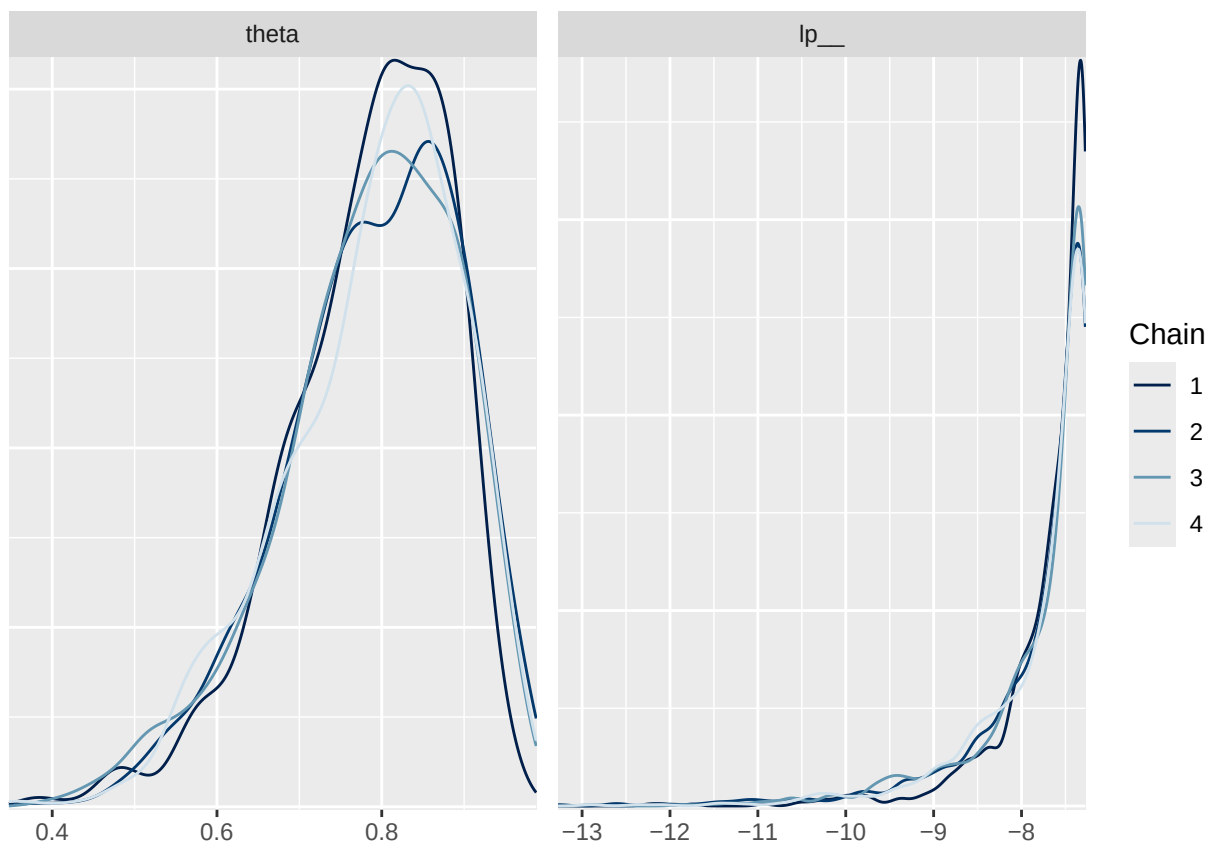
And here he have a plot for the first 100 iterations (including warmup period)

```
draws <- as.array(bb_sim)
draws_100 <- draws[1:100,,] # subset first 100 iterations
mcmc_trace(draws_100)
```



The trace plot for the first 100 iterations (after warm up period) features little to no trend, heteroscedasticity, or any other indication that the chain has not reached its stationary distribution. This suggests that the warmup period was adequate, and that the Markov Chain has converged.

```
mcmc_dens_overlay(bb_sim)
```



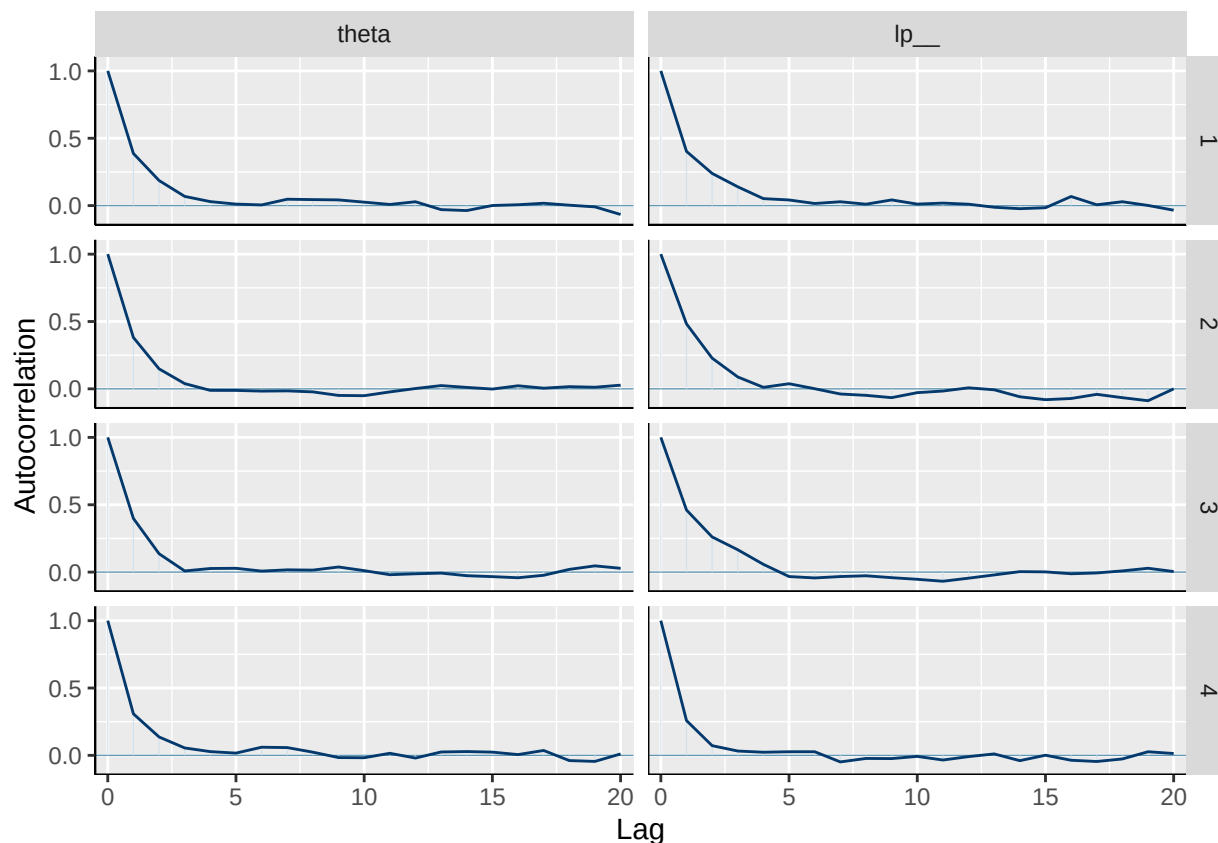
Based on the overlay plot of the four chains, we see the resulting posterior distribution is roughly the same across all four chains. Although there are some notable deviations in the mode or skew, they all appear to closely approximate the correct beta distribution.

```
neff_ratio(bb_sim)
```

```
##      theta      lp__
## 0.4403235 0.3868069
```

Based on the `neff_ratio` above, we have that the ratio of the effective sample size estimate to total sample size. The ratio is calculated to be around 0.36741, indicating that we have an effective sample size of about 1468 (based on 4000 total samples).

```
mcmc_acf(bb_sim)
```



The acf plot above indicates that the autocorrelation between mcmc samples decays rapidly/exponentially with the lag. This means that although consecutive samples, or samples that are close to one another in time may be highly correlated with one another, this dependence decays significantly as the lag gets larger. A more rapidly decaying acf is suggestive of a shorter mixing time. A faster mixing time corresponds to a larger effective sample size (and vice versa) since the effective sample size is smaller if the samples are more correlated with one

Question 2 (b): (Note that I use lambda as the parameter instead of theta to distinguish from part (a))

```
y = c(2, 8)
pg_sim <- stan(
  file = "578 HW 5 Q2b.stan",
  data = list(y = y, N = length(y)),
  chains = 4, iter = 2000, warmup = 1000, thin = 1)
```

```
## Warning in readLines(file, warn = TRUE): incomplete final line found on
## 'C:\Users\njp09\Documents\Notes\6 - Fall 2025\Bayesian Stats\578 HW 5 Q2b.stan'

##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 1).
## Chain 1:
## Chain 1: Gradient evaluation took 2.9e-05 seconds
## Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 0.29 seconds.
## Chain 1: Adjust your expectations accordingly!
## Chain 1:
## Chain 1:
```

```

## Chain 1: Iteration:    1 / 2000 [ 0%] (Warmup)
## Chain 1: Iteration:   200 / 2000 [ 10%] (Warmup)
## Chain 1: Iteration:   400 / 2000 [ 20%] (Warmup)
## Chain 1: Iteration:   600 / 2000 [ 30%] (Warmup)
## Chain 1: Iteration:   800 / 2000 [ 40%] (Warmup)
## Chain 1: Iteration:  1000 / 2000 [ 50%] (Warmup)
## Chain 1: Iteration: 1001 / 2000 [ 50%] (Sampling)
## Chain 1: Iteration:  1200 / 2000 [ 60%] (Sampling)
## Chain 1: Iteration:  1400 / 2000 [ 70%] (Sampling)
## Chain 1: Iteration:  1600 / 2000 [ 80%] (Sampling)
## Chain 1: Iteration:  1800 / 2000 [ 90%] (Sampling)
## Chain 1: Iteration:  2000 / 2000 [100%] (Sampling)
## Chain 1:
## Chain 1: Elapsed Time: 0.028 seconds (Warm-up)
## Chain 1:                0.024 seconds (Sampling)
## Chain 1:                0.052 seconds (Total)
## Chain 1:
##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 2).
## Chain 2:
## Chain 2: Gradient evaluation took 6e-06 seconds
## Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 0.06 seconds.
## Chain 2: Adjust your expectations accordingly!
## Chain 2:
## Chain 2:
## Chain 2: Iteration:    1 / 2000 [ 0%] (Warmup)
## Chain 2: Iteration:   200 / 2000 [ 10%] (Warmup)
## Chain 2: Iteration:   400 / 2000 [ 20%] (Warmup)
## Chain 2: Iteration:   600 / 2000 [ 30%] (Warmup)
## Chain 2: Iteration:   800 / 2000 [ 40%] (Warmup)
## Chain 2: Iteration:  1000 / 2000 [ 50%] (Warmup)
## Chain 2: Iteration: 1001 / 2000 [ 50%] (Sampling)
## Chain 2: Iteration:  1200 / 2000 [ 60%] (Sampling)
## Chain 2: Iteration:  1400 / 2000 [ 70%] (Sampling)
## Chain 2: Iteration:  1600 / 2000 [ 80%] (Sampling)
## Chain 2: Iteration:  1800 / 2000 [ 90%] (Sampling)
## Chain 2: Iteration:  2000 / 2000 [100%] (Sampling)
## Chain 2:
## Chain 2: Elapsed Time: 0.019 seconds (Warm-up)
## Chain 2:                0.017 seconds (Sampling)
## Chain 2:                0.036 seconds (Total)
## Chain 2:
##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 3).
## Chain 3:
## Chain 3: Gradient evaluation took 8e-06 seconds
## Chain 3: 1000 transitions using 10 leapfrog steps per transition would take 0.08 seconds.
## Chain 3: Adjust your expectations accordingly!
## Chain 3:
## Chain 3:
## Chain 3: Iteration:    1 / 2000 [ 0%] (Warmup)
## Chain 3: Iteration:   200 / 2000 [ 10%] (Warmup)
## Chain 3: Iteration:   400 / 2000 [ 20%] (Warmup)
## Chain 3: Iteration:   600 / 2000 [ 30%] (Warmup)

```

```

## Chain 3: Iteration: 800 / 2000 [ 40%] (Warmup)
## Chain 3: Iteration: 1000 / 2000 [ 50%] (Warmup)
## Chain 3: Iteration: 1001 / 2000 [ 50%] (Sampling)
## Chain 3: Iteration: 1200 / 2000 [ 60%] (Sampling)
## Chain 3: Iteration: 1400 / 2000 [ 70%] (Sampling)
## Chain 3: Iteration: 1600 / 2000 [ 80%] (Sampling)
## Chain 3: Iteration: 1800 / 2000 [ 90%] (Sampling)
## Chain 3: Iteration: 2000 / 2000 [100%] (Sampling)
## Chain 3:
## Chain 3: Elapsed Time: 0.023 seconds (Warm-up)
## Chain 3: 0.014 seconds (Sampling)
## Chain 3: 0.037 seconds (Total)
## Chain 3:
##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 4).
## Chain 4:
## Chain 4: Gradient evaluation took 6e-06 seconds
## Chain 4: 1000 transitions using 10 leapfrog steps per transition would take 0.06 seconds.
## Chain 4: Adjust your expectations accordingly!
## Chain 4:
## Chain 4:
## Chain 4: Iteration: 1 / 2000 [ 0%] (Warmup)
## Chain 4: Iteration: 200 / 2000 [ 10%] (Warmup)
## Chain 4: Iteration: 400 / 2000 [ 20%] (Warmup)
## Chain 4: Iteration: 600 / 2000 [ 30%] (Warmup)
## Chain 4: Iteration: 800 / 2000 [ 40%] (Warmup)
## Chain 4: Iteration: 1000 / 2000 [ 50%] (Warmup)
## Chain 4: Iteration: 1001 / 2000 [ 50%] (Sampling)
## Chain 4: Iteration: 1200 / 2000 [ 60%] (Sampling)
## Chain 4: Iteration: 1400 / 2000 [ 70%] (Sampling)
## Chain 4: Iteration: 1600 / 2000 [ 80%] (Sampling)
## Chain 4: Iteration: 1800 / 2000 [ 90%] (Sampling)
## Chain 4: Iteration: 2000 / 2000 [100%] (Sampling)
## Chain 4:
## Chain 4: Elapsed Time: 0.015 seconds (Warm-up)
## Chain 4: 0.014 seconds (Sampling)
## Chain 4: 0.029 seconds (Total)
## Chain 4:

```

```

posterior_lambda <- as.array(pg_sim, pars = "lambda")
dim(posterior_theta)

```

```

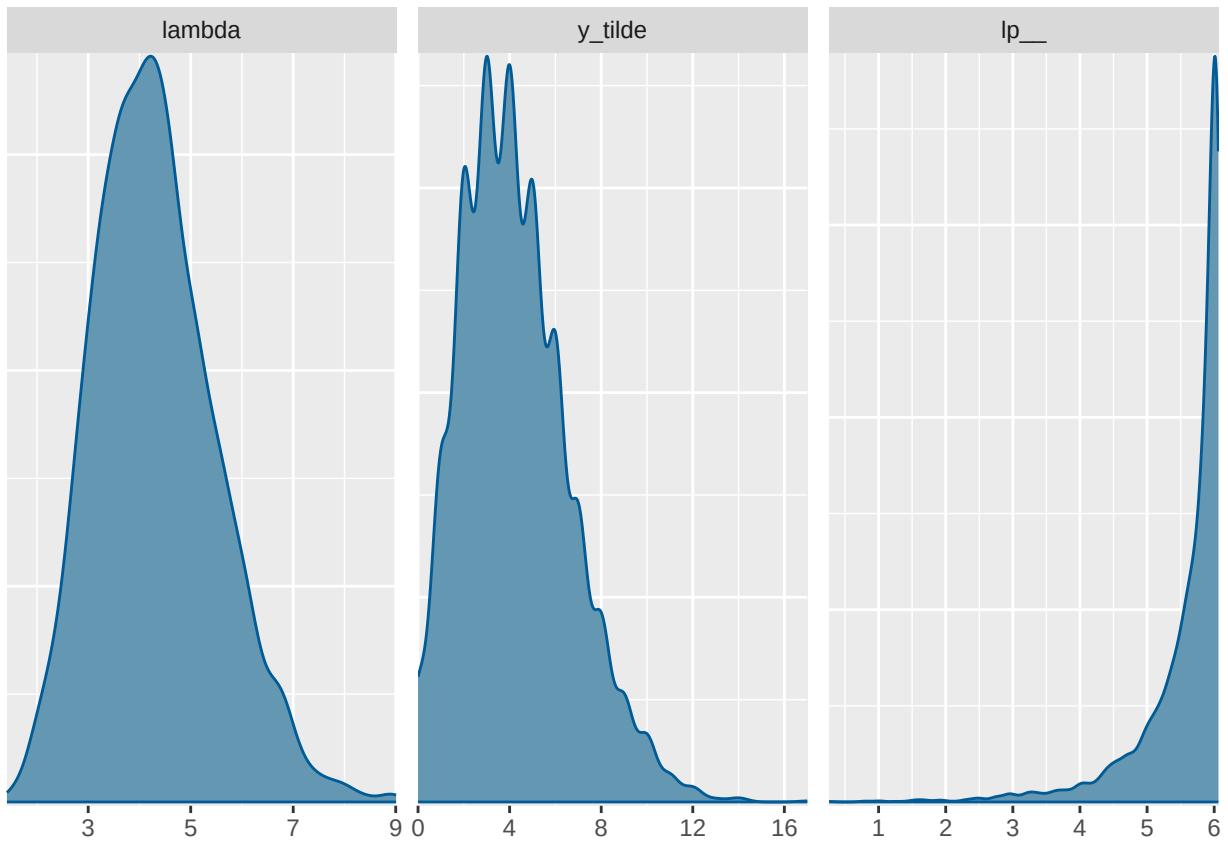
## [1] 1000 4 1

```

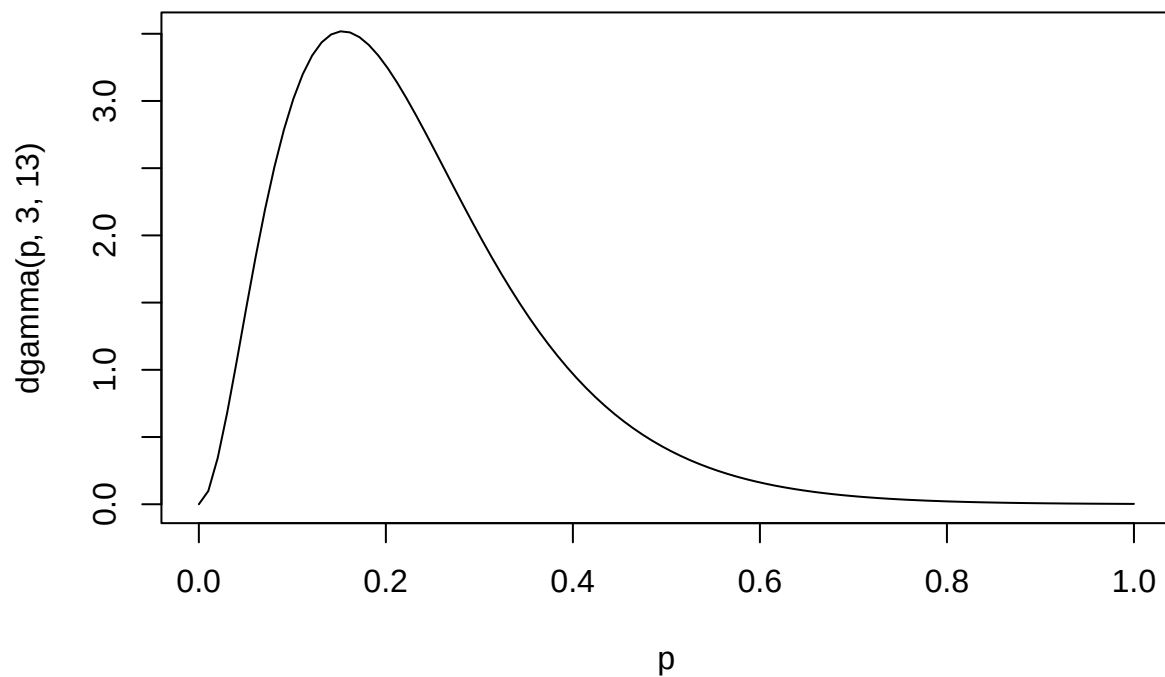
```

mcmc_dens(pg_sim)

```



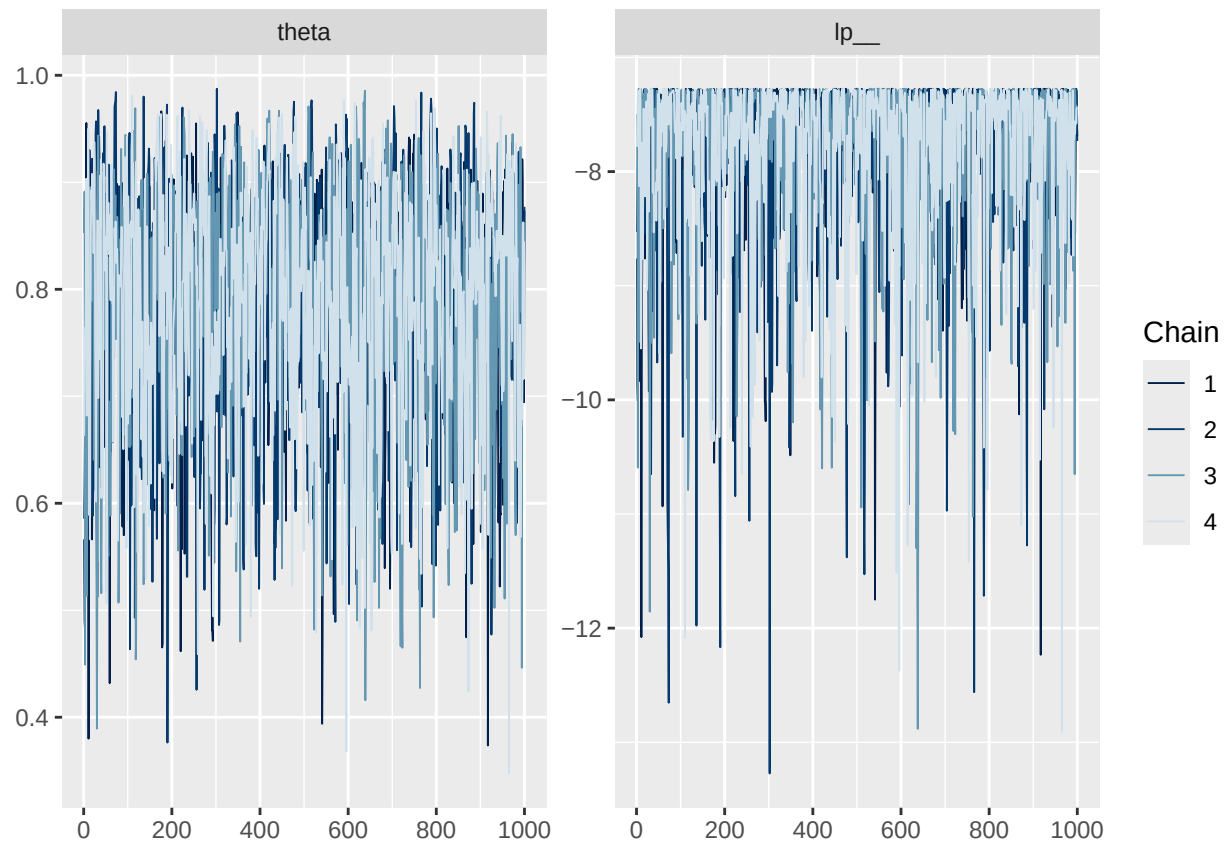
```
p = seq(0, 1, length=100)
plot(p, dgamma(p, 3, 13), type='l')
```



Once again, we see that the posterior distribution from the Stan model matches closely the expected Gamma density. As we shall see with the result below, the analysis from the binomial model also applies to the results here:

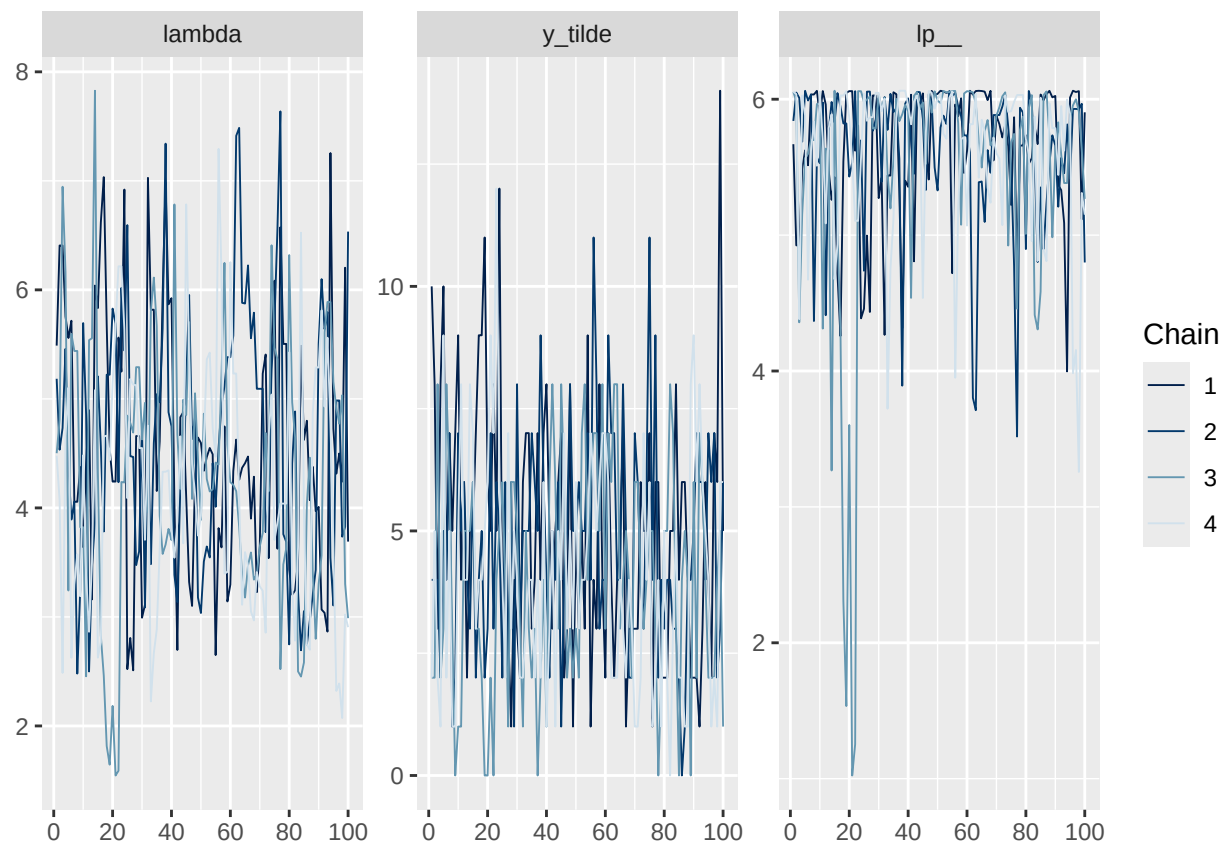
Below is a trace plot for all 4 MCMC chains

```
mcmc_trace(bb_sim)
```

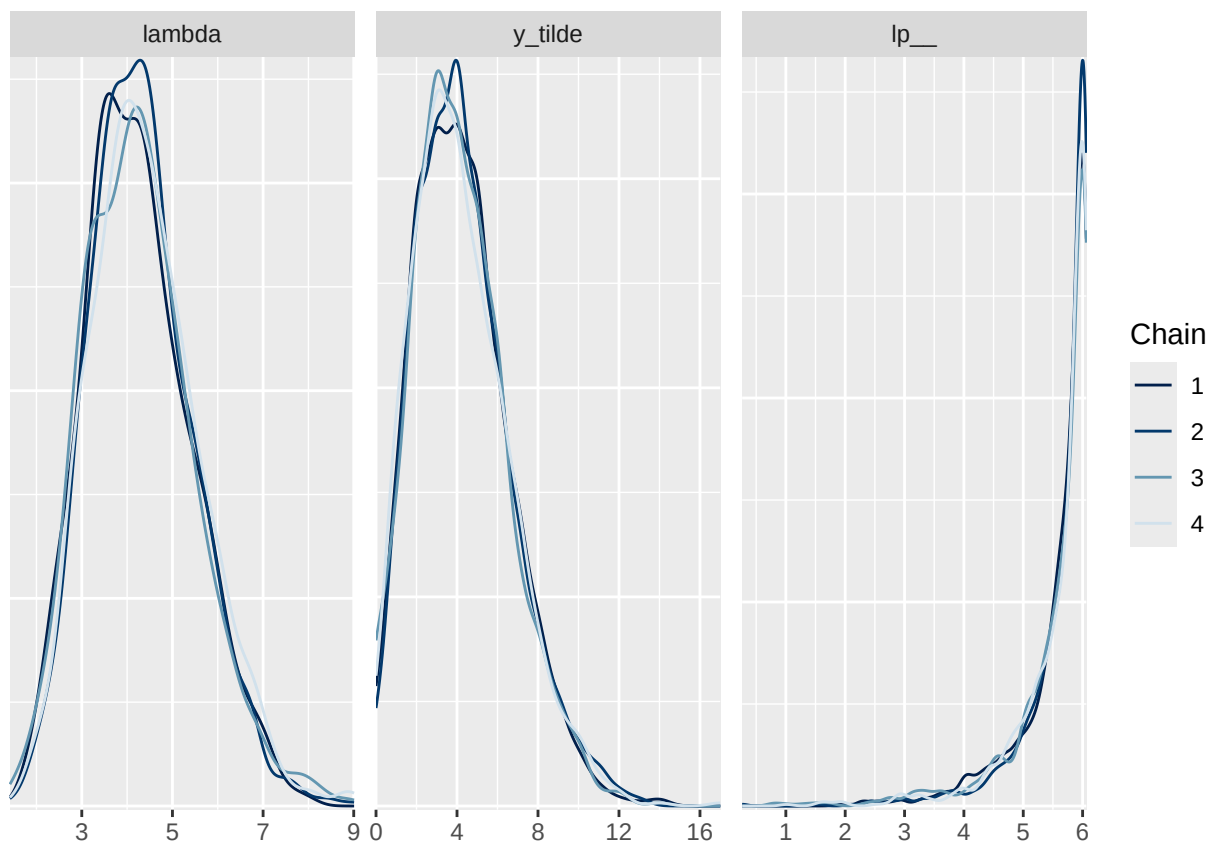


And here he have a plot for the first 100 iterations (including warmup period)

```
draws <- as.array(pg_sim)
draws_100 <- draws[1:100,,] # subset first 100 iterations
mcmc_trace(draws_100)
```



```
mcmc_dens_overlay(pg_sim)
```

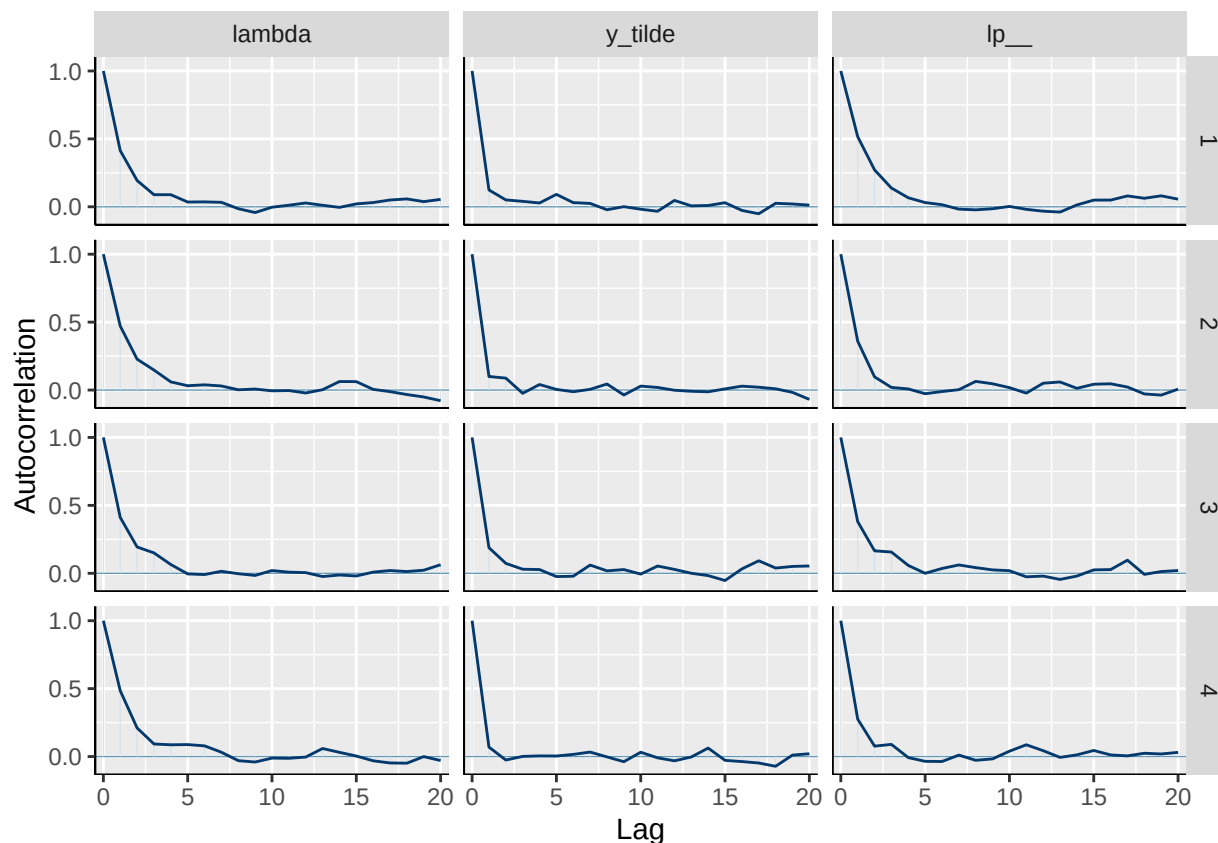


```
neff_ratio(bb_sim)
```

```
##      theta      lp__
## 0.4403235 0.3868069
```

Based on the `neff_ratio` above, we have that the ratio of the effective sample size estimate to total sample size. The ratio is calculated to be around 0.36741, indicating that we have an effective sample size of about 1468 (based on 4000 total samples).

```
mcmc_acf(pg_sim)
```



The acf plot above indicates that the autocorrelation between mcmc samples decays rapidly/exponentially with the lag. This means that although consecutive samples, or samples that are close to one another in time may be highly correlated with one another, this dependence decays significantly as the lag gets larger. A more rapidly decaying acf is suggestive of a shorter mixing time. A faster mixing time corresponds to a larger effective sample size (and vice versa) since the effective sample size is smaller if the samples are more correlated with one another.

Question 3 (a): Below is an implementation of the Metropolis Random Walk:

```
# init
dat = read.csv("bioassay.csv")
y = dat$deaths
n = dat$n
x = dat$logdose
gb <- glm(cbind(y, n - y) ~ x, family = binomial)
alpha <- coef(gb)[1]
beta <- coef(gb)[2]

# define density functions for proposal evaluation
sampling_density = function(yi, ni, xi, a, b){

  p = (exp(a + b*xi)/(1 + exp(a + b*xi)))
  result = p^yi * (1-p)^(ni-yi)
  return(result)
}

posterior_density = function(a, b){
```

```

result = 1
for(i in seq(1,4)){
  result = result*sampling_density(y[i], n[i], x[i], a, b)
}
return(result)
}

# run Metropolis (2 Chains)

iters=10000
sigma_a = 0.7 # step size parameter
sigma_b = 1.3

mc_alphas = c(alpha)
mc_betas = c(beta)
for(s in seq(1,iters)){

  # update proposal
  alpha_prime = alpha + sigma_a*rnorm(1)
  beta_prime = beta + sigma_b*rnorm(1)

  # calculate acceptance
  A = exp(log(posterior_density(alpha_prime, beta_prime)) - log(posterior_density(alpha, beta)))

  # accept/reject
  u = runif(1)
  if(u < A){
    alpha = alpha_prime
    beta = beta_prime

    mc_alphas = c(mc_alphas, alpha_prime)
    mc_betas = c(mc_betas, beta_prime)
  }
  else{
    mc_alphas = c(mc_alphas, alpha)
    mc_betas = c(mc_betas, beta)
  }
}

means = c(mean(mc_alphas), mean(mc_betas))
stddevs = c(sd(mc_alphas), sd(mc_betas))

cbind(means, stddevs)

##           means  stddevs
## [1,]  1.325913  1.075404
## [2,] 11.325445  5.199954

```

Based on the metropolis algorithm, the posterior expectations for alpha and beta are 1.185 and 10.796 respectively. The standard deviations for alpha and beta are 0.997 and 5.068 respectively.

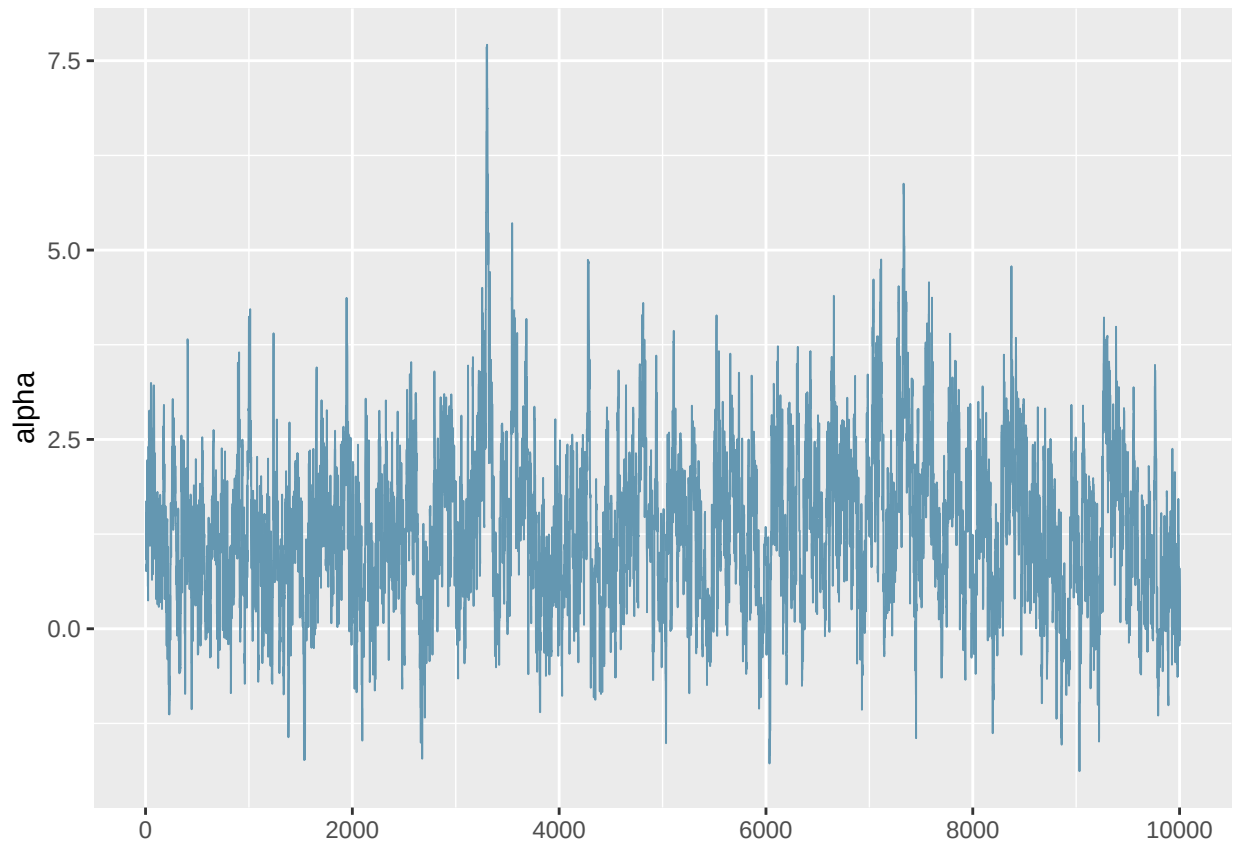
Question 3(b): Below are the diagnostic plots for the Metropolis Random Walk:

```
iterations = seq(1, iters+1)

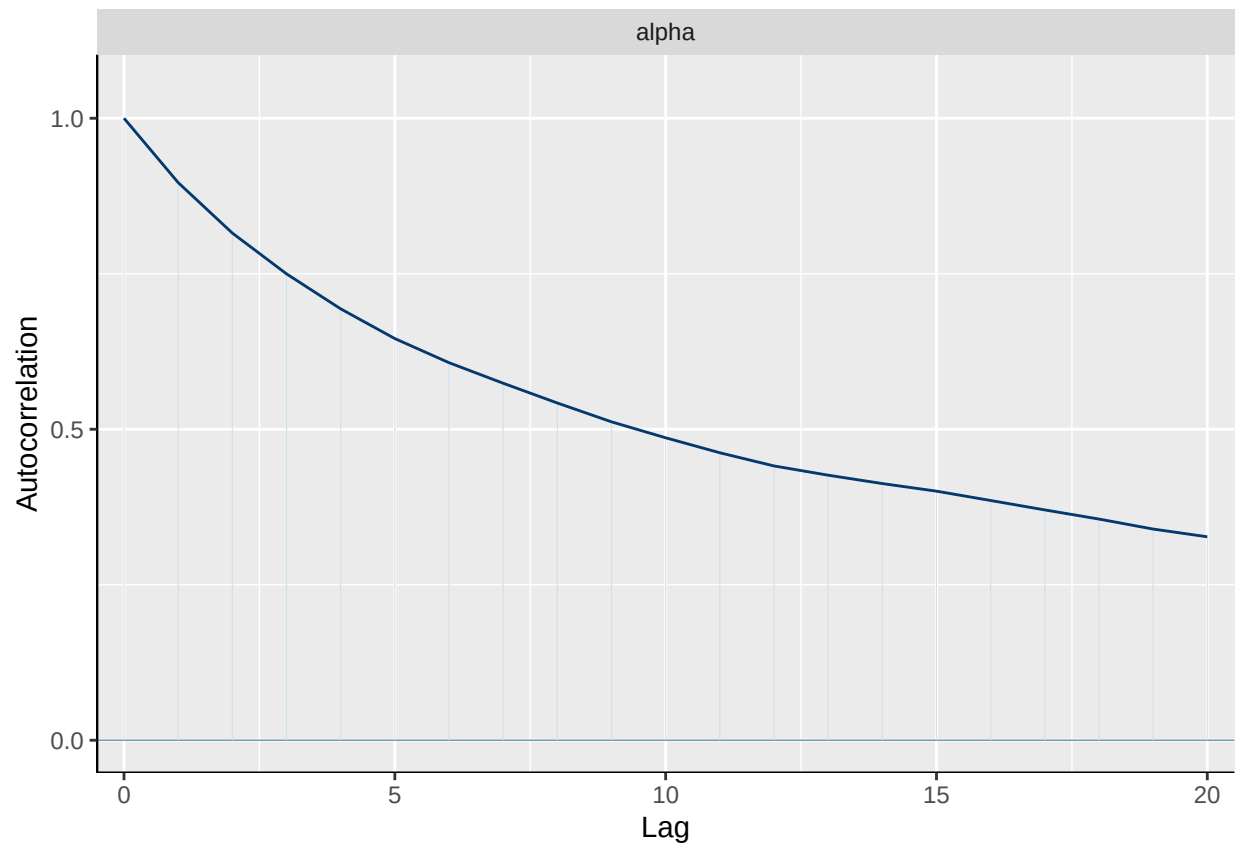
mc_alphas = matrix(mc_alphas, ncol = 1)
mc_betas = matrix(mc_betas, ncol = 1 )

colnames(mc_alphas) = "alpha"
colnames(mc_betas) = "beta"

mcmc_trace(mc_alphas)
```

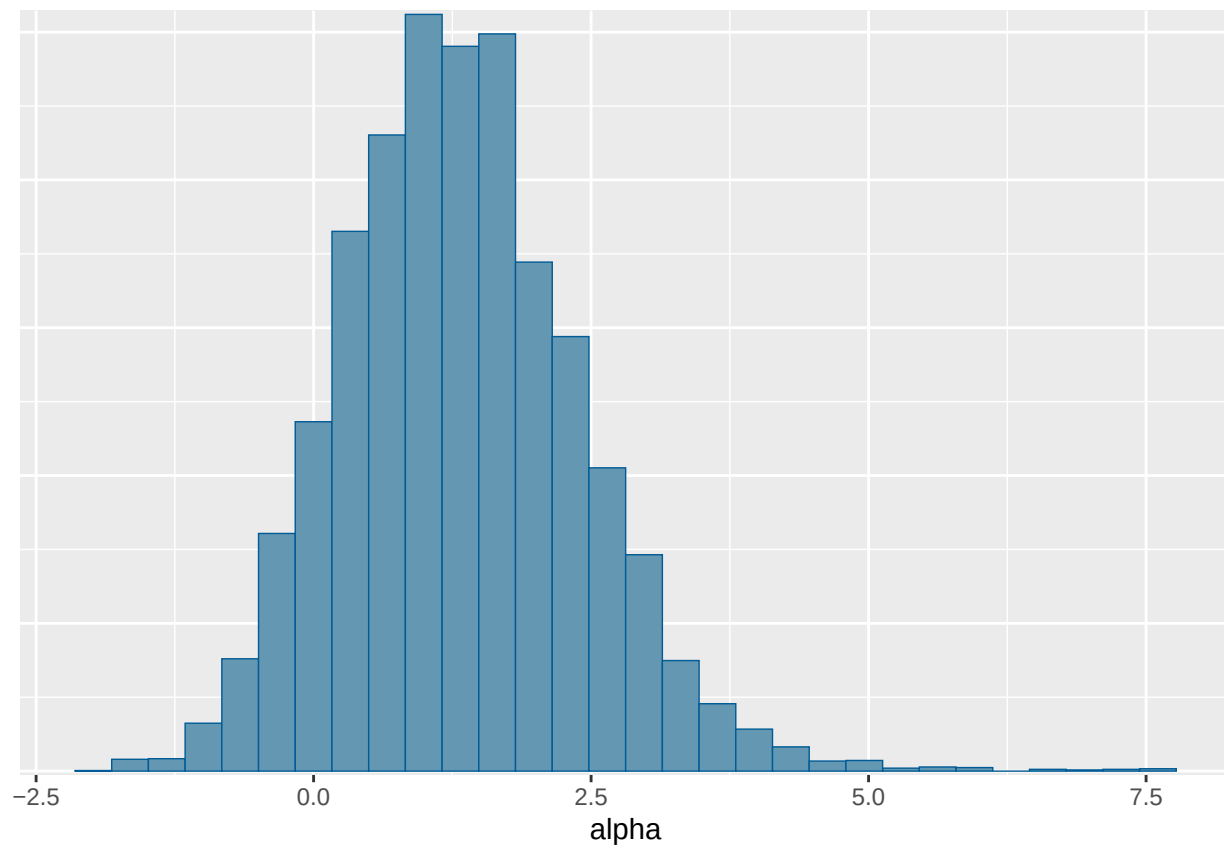


```
mcmc_acf(mc_alphas)
```

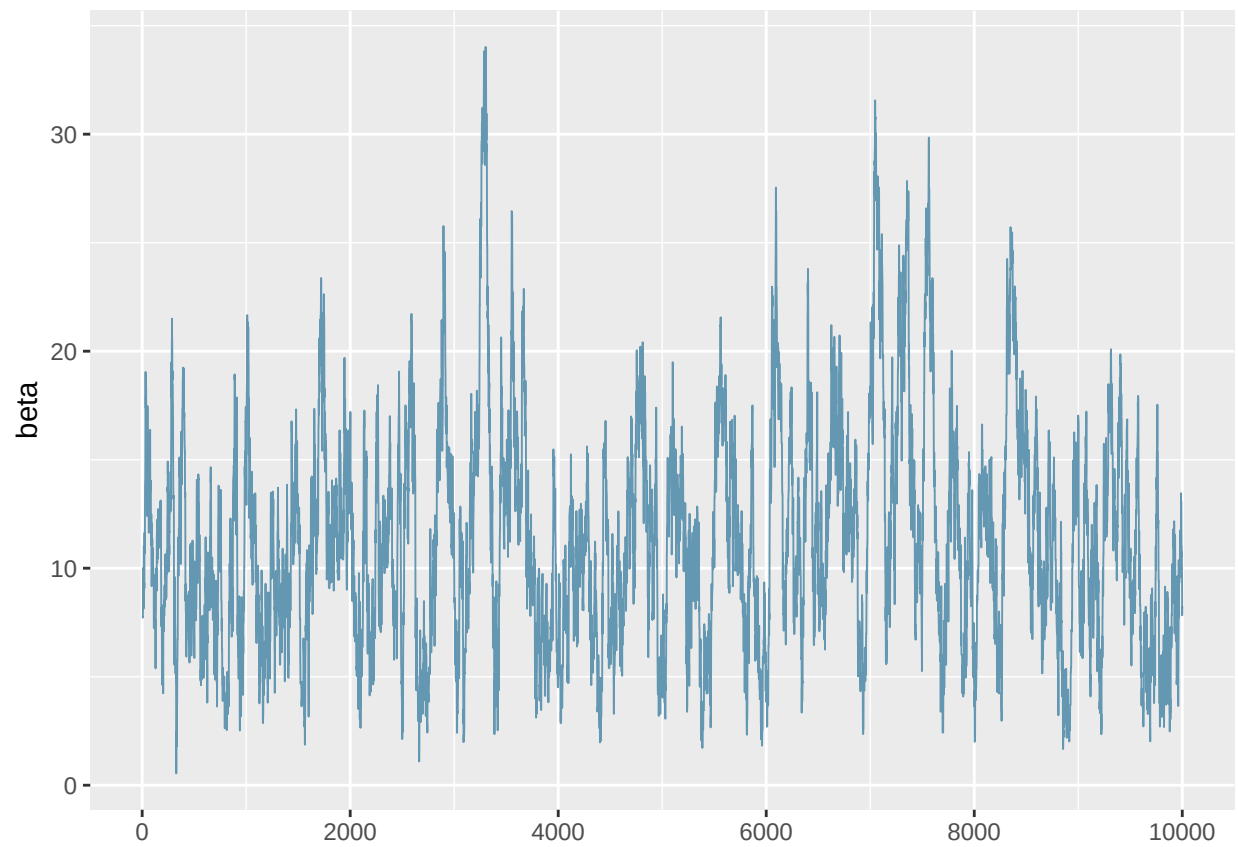


```
mcmc_hist(mc_alphas)
```

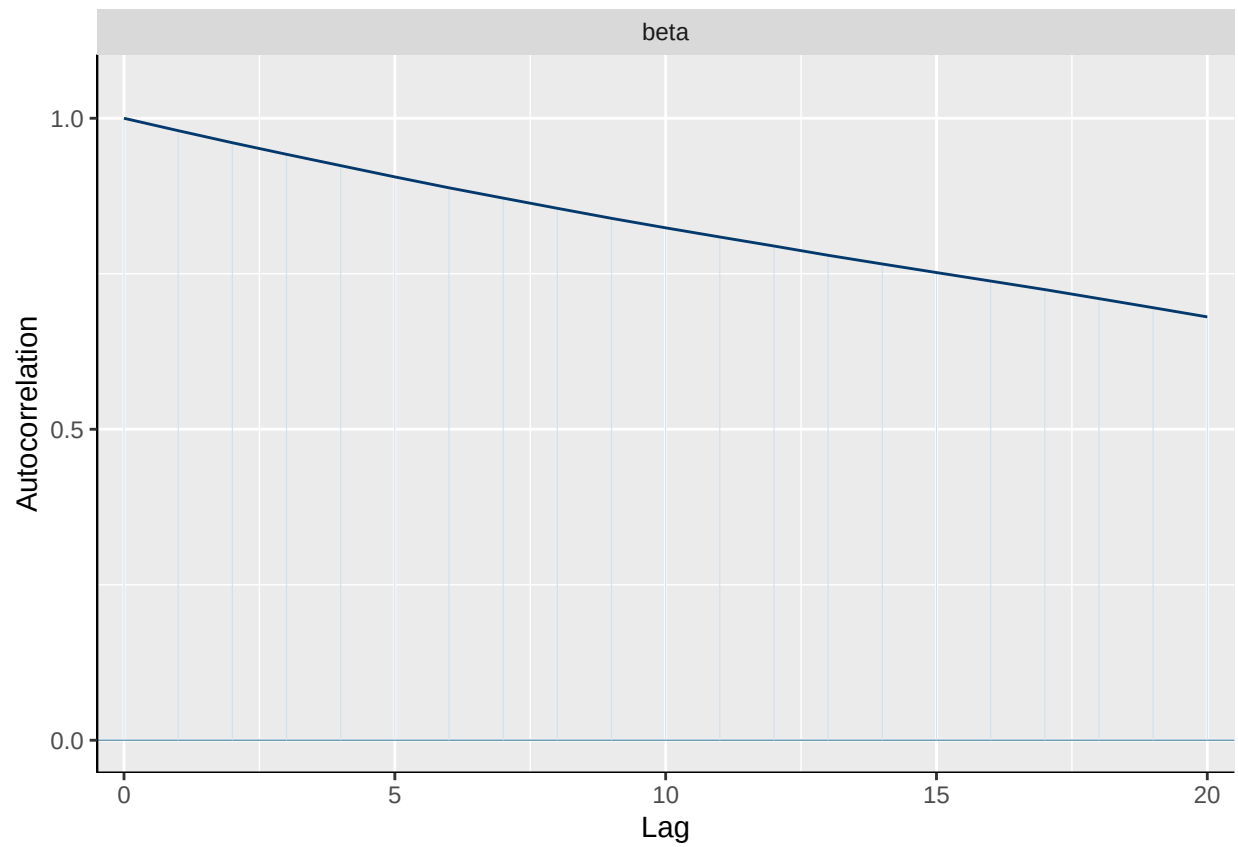
```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



```
mcmc_trace(mc_betas)
```

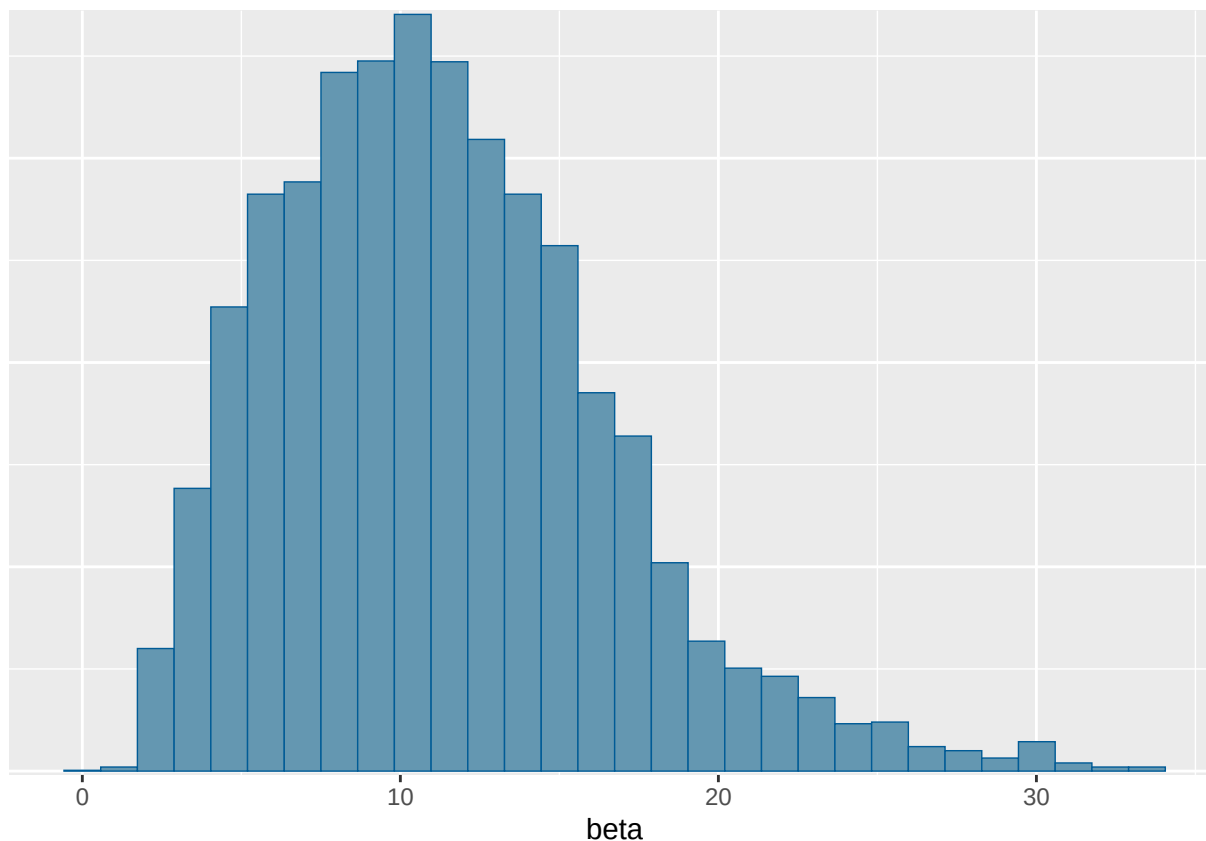


```
mcmc_acf(mc_betas)
```



```
mcmc_hist(mc_betas)
```

```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



Based on the diagnostic plots, we see that the metropolis algorithm appears to work well for estimating alpha: the acf decays rapidly and there is no clear trend in the trace plot. On the other hand, it performs poorly due to a stronger autocorrelation which can be seen by local trends in the trace plot and a slowly decaying acf. The poor performance is likely due to the choice of step size for the random walk process, however fine tuning is difficult in this scenario since creating too large a step size for beta causes too frequent a rejection rate.

Question 3(c) Below is an implementation of a NUTS algorithm via Stan

```
lr_sim = stan(
  file = "578 HW 5 Q3.stan",
  data = list(N = 4, n = n, y = y, x = x),
  chains = 4, iter = 2000, warmup = 1000, thin = 1)
```

```
## Warning in readLines(file, warn = TRUE): incomplete final line found on
## 'C:\Users\njp09\Documents\Notes\6 - Fall 2025\Bayesian Stats\578 HW 5 Q3.stan'

##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 1).
## Chain 1:
## Chain 1: Gradient evaluation took 4.8e-05 seconds
## Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 0.48 seconds.
## Chain 1: Adjust your expectations accordingly!
## Chain 1:
## Chain 1:
## Chain 1: Iteration:    1 / 2000 [  0%]  (Warmup)
```

```

## Chain 1: Iteration: 200 / 2000 [ 10%] (Warmup)
## Chain 1: Iteration: 400 / 2000 [ 20%] (Warmup)
## Chain 1: Iteration: 600 / 2000 [ 30%] (Warmup)
## Chain 1: Iteration: 800 / 2000 [ 40%] (Warmup)
## Chain 1: Iteration: 1000 / 2000 [ 50%] (Warmup)
## Chain 1: Iteration: 1001 / 2000 [ 50%] (Sampling)
## Chain 1: Iteration: 1200 / 2000 [ 60%] (Sampling)
## Chain 1: Iteration: 1400 / 2000 [ 70%] (Sampling)
## Chain 1: Iteration: 1600 / 2000 [ 80%] (Sampling)
## Chain 1: Iteration: 1800 / 2000 [ 90%] (Sampling)
## Chain 1: Iteration: 2000 / 2000 [100%] (Sampling)
## Chain 1:
## Chain 1: Elapsed Time: 0.047 seconds (Warm-up)
## Chain 1: 0.034 seconds (Sampling)
## Chain 1: 0.081 seconds (Total)
## Chain 1:
##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 2).
## Chain 2:
## Chain 2: Gradient evaluation took 1e-05 seconds
## Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 0.1 seconds.
## Chain 2: Adjust your expectations accordingly!
## Chain 2:
## Chain 2:
## Chain 2: Iteration: 1 / 2000 [ 0%] (Warmup)
## Chain 2: Iteration: 200 / 2000 [ 10%] (Warmup)
## Chain 2: Iteration: 400 / 2000 [ 20%] (Warmup)
## Chain 2: Iteration: 600 / 2000 [ 30%] (Warmup)
## Chain 2: Iteration: 800 / 2000 [ 40%] (Warmup)
## Chain 2: Iteration: 1000 / 2000 [ 50%] (Warmup)
## Chain 2: Iteration: 1001 / 2000 [ 50%] (Sampling)
## Chain 2: Iteration: 1200 / 2000 [ 60%] (Sampling)
## Chain 2: Iteration: 1400 / 2000 [ 70%] (Sampling)
## Chain 2: Iteration: 1600 / 2000 [ 80%] (Sampling)
## Chain 2: Iteration: 1800 / 2000 [ 90%] (Sampling)
## Chain 2: Iteration: 2000 / 2000 [100%] (Sampling)
## Chain 2:
## Chain 2: Elapsed Time: 0.045 seconds (Warm-up)
## Chain 2: 0.037 seconds (Sampling)
## Chain 2: 0.082 seconds (Total)
## Chain 2:
##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 3).
## Chain 3:
## Chain 3: Gradient evaluation took 9e-06 seconds
## Chain 3: 1000 transitions using 10 leapfrog steps per transition would take 0.09 seconds.
## Chain 3: Adjust your expectations accordingly!
## Chain 3:
## Chain 3:
## Chain 3: Iteration: 1 / 2000 [ 0%] (Warmup)
## Chain 3: Iteration: 200 / 2000 [ 10%] (Warmup)
## Chain 3: Iteration: 400 / 2000 [ 20%] (Warmup)
## Chain 3: Iteration: 600 / 2000 [ 30%] (Warmup)
## Chain 3: Iteration: 800 / 2000 [ 40%] (Warmup)

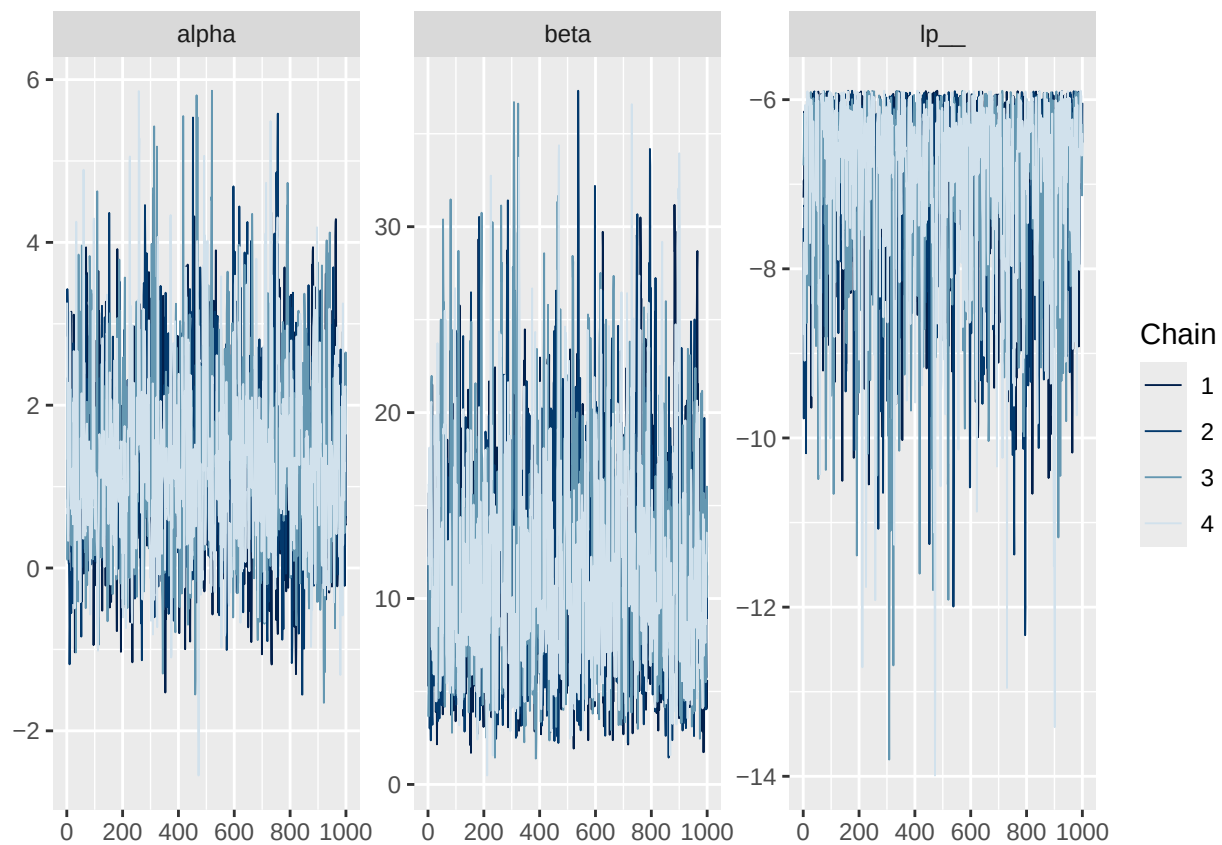
```

```

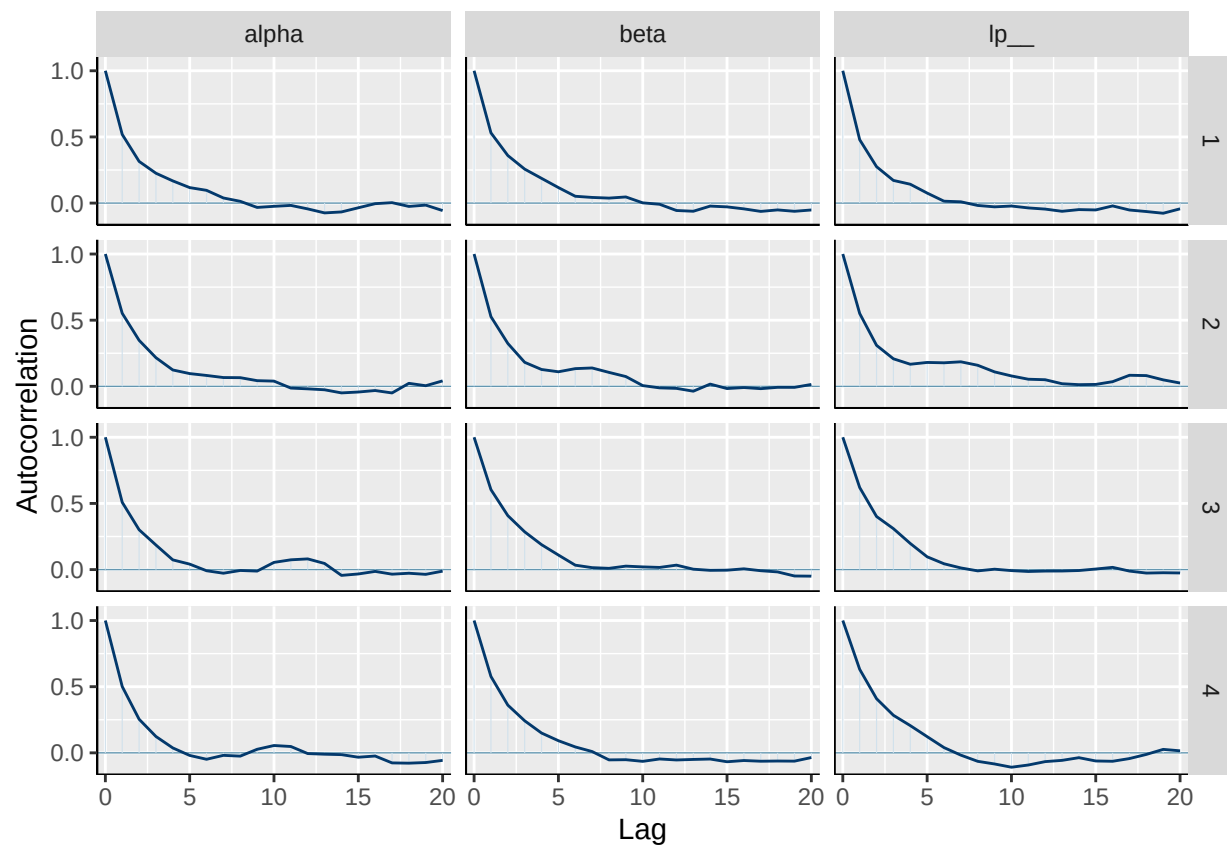
## Chain 3: Iteration: 1000 / 2000 [ 50%] (Warmup)
## Chain 3: Iteration: 1001 / 2000 [ 50%] (Sampling)
## Chain 3: Iteration: 1200 / 2000 [ 60%] (Sampling)
## Chain 3: Iteration: 1400 / 2000 [ 70%] (Sampling)
## Chain 3: Iteration: 1600 / 2000 [ 80%] (Sampling)
## Chain 3: Iteration: 1800 / 2000 [ 90%] (Sampling)
## Chain 3: Iteration: 2000 / 2000 [100%] (Sampling)
## Chain 3:
## Chain 3: Elapsed Time: 0.044 seconds (Warm-up)
## Chain 3:           0.046 seconds (Sampling)
## Chain 3:           0.09 seconds (Total)
## Chain 3:
##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 4).
## Chain 4:
## Chain 4: Gradient evaluation took 9e-06 seconds
## Chain 4: 1000 transitions using 10 leapfrog steps per transition would take 0.09 seconds.
## Chain 4: Adjust your expectations accordingly!
## Chain 4:
## Chain 4:
## Chain 4: Iteration:    1 / 2000 [  0%] (Warmup)
## Chain 4: Iteration:   200 / 2000 [ 10%] (Warmup)
## Chain 4: Iteration:   400 / 2000 [ 20%] (Warmup)
## Chain 4: Iteration:   600 / 2000 [ 30%] (Warmup)
## Chain 4: Iteration:   800 / 2000 [ 40%] (Warmup)
## Chain 4: Iteration:  1000 / 2000 [ 50%] (Warmup)
## Chain 4: Iteration:  1001 / 2000 [ 50%] (Sampling)
## Chain 4: Iteration:  1200 / 2000 [ 60%] (Sampling)
## Chain 4: Iteration:  1400 / 2000 [ 70%] (Sampling)
## Chain 4: Iteration:  1600 / 2000 [ 80%] (Sampling)
## Chain 4: Iteration:  1800 / 2000 [ 90%] (Sampling)
## Chain 4: Iteration:  2000 / 2000 [100%] (Sampling)
## Chain 4:
## Chain 4: Elapsed Time: 0.042 seconds (Warm-up)
## Chain 4:           0.045 seconds (Sampling)
## Chain 4:           0.087 seconds (Total)
## Chain 4:

```

```
mcmc_trace(lr_sim)
```

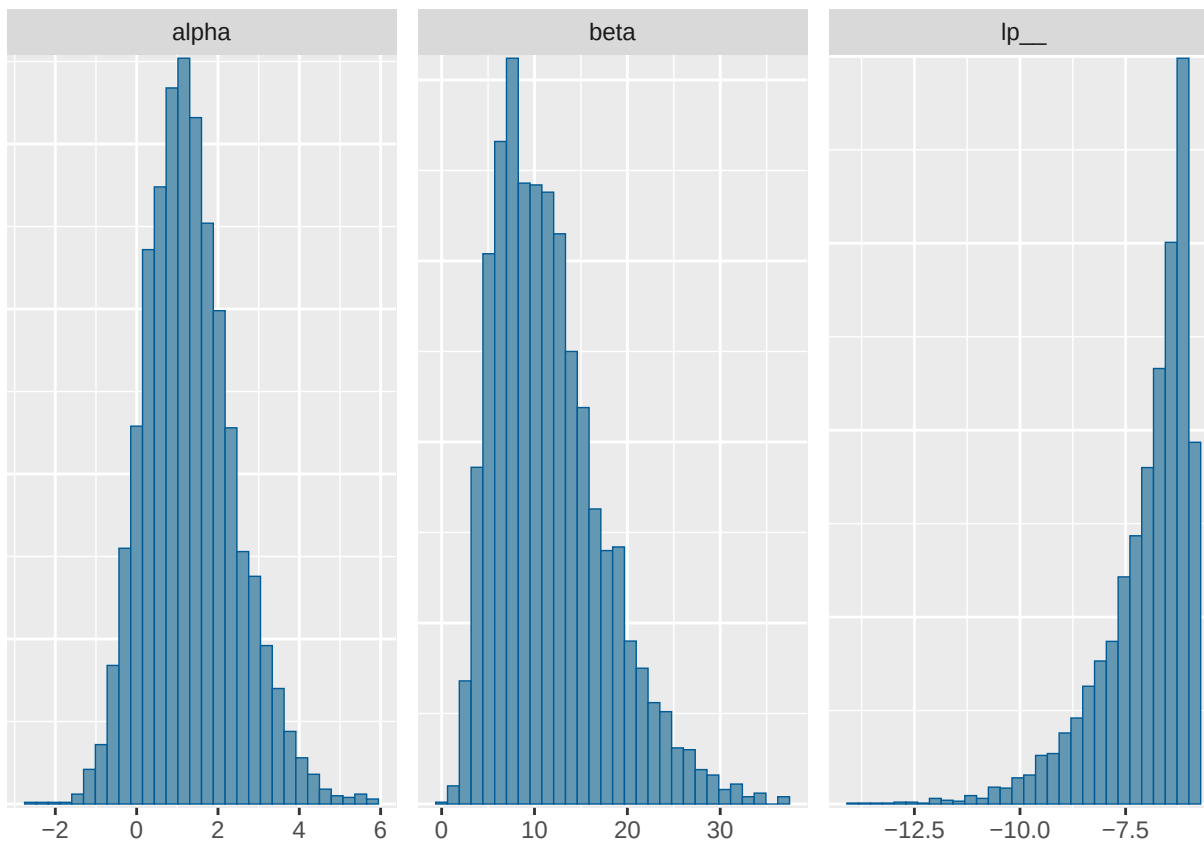


```
mcmc_acf(lr_sim)
```



```
mcmc_hist(lr_sim)
```

```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



The stan model appears to outperform the metropolis algorithm for both parameters, and especially for inference of beta. We see the ACF decays rapidly and there are no clear trends in the trace plot of either parameter. In addition, the posterior distributions look comparable to the ones from the metropolis algorithm, a Gaussian with some rightward skew.

```
posterior_alpha <- as.array(lr_sim, pars = "alpha")
posterior_beta  <- as.array(lr_sim, pars = "beta")
```

```
mean(posterior_alpha)
```

```
## [1] 1.313374
```

```
mean(posterior_beta)
```

```
## [1] 11.65703
```

```
sd(posterior_alpha)
```

```
## [1] 1.103834
```

```
sd(posterior_beta)
```

```
## [1] 5.899709
```

Based on the stan model, the posterior expectations for alpha and beta are 1.278 and 11.341 respectively. The standard deviations for alpha and beta are 1.079 and 5.576 respectively.